

UEnv — 下一代分布式训练环境框架方案-v7.1

版本: v7.1 | 日期: 2026-04-27 | 状态: 设计评审

目录

- 一、产品概述
 - 1.1 项目背景
 - 1.2 产品定位
 - 1.3 目标用户
 - 1.4 核心价值主张
 - 1.5 术语定义
- 二、系统设计总览
 - 2.1 总体架构全貌
 - 2.2 设计思路
 - 2.3 系统分层架构
 - 2.4 子系统协作关系
- 三、场景与需求
 - 3.1 用户故事
 - 3.2 功能需求清单
 - 3.3 非功能需求
 - 3.4 约束与假设
- 四、环境构造方法
 - 4.1 环境定义
 - 4.2 环境能力扩展
 - 4.3 奖励系统
 - 4.4 环境执行模式
 - 4.5 环境容器化
 - 4.6 环境注册与分发

- 4.7 内置环境策略
- [五、交互机制](#五交互机制)
 - 5.1 通信架构选型
 - 5.2 通信协议
 - 5.3 gRPC Service 设计
 - 5.4 Training Adapter 层
 - 5.5 Episode 完整交互时序
 - 5.6 协议版本策略
 - 5.7 流控与背压机制
 - 5.8 消息大小与传输策略
 - 5.9 AdminService
 - 5.10 协议类型安全改进
 - 5.11 批量提交语义 (SubmitBatch)
 - 5.12 异步提交模式 (SubmitEpisodeAsync)
 - 5.13 通信模式对比
- 六、任务调度](#六任务调度)
 - 6.1 调度器框架
 - 6.2 调度流程
 - 6.3 调度策略
 - 6.4 环境预热池
 - 6.5 扩缩容策略
 - 6.6 UEnv Server 水平扩展
 - 6.7 自适应心跳
 - 6.8 Server 性能建模
 - 6.9 分阶段状态管理策略
- 七、运行时管理
 - 7.1 Worker 架构
 - 7.2 后端引擎
 - 7.3 渐进部署路径
- 八、数据结构
 - 8.1 Server 内部数据与 RewardConfig

- 8.2 数据实体关系图
- 8.3 Protobuf 类型安全改进
- 8.4 错误码枚举
- 九、状态机
 - 9.1 Episode 状态机
 - 9.2 环境实例状态机
 - 9.3 Worker 生命周期状态机
 - 9.4 UEnv Server 启动/关闭状态机
- 十、容错与可靠性
 - 10.1 四层容错金字塔
 - 10.2 重试策略
 - 10.3 数据完整性
 - 10.4 UEnv Server 高可用
 - 10.5 网络分区处理
 - 10.6 脑裂防护
 - 10.7 Worker 断连策略
 - 10.8 请求持久化
 - 10.9 GC 暂停与心跳隔离
 - 10.10 架构评审：缺陷严重度修正
- 十一、性能分析
 - 11.1 延迟预算
 - 11.2 吞吐量估算
- 十二、使用场景
 - 12.1 数学训练环境（ROLL + MathEnv）
 - 12.2 代码训练环境（VeRL + CodeEnv）
 - 12.3 Agent 训练环境（NeMo-RL + AgentEnv）
 - 12.4 自定义环境开发与发布
 - 12.5 大规模并行训练
- 十三、演进路线
 - 13.1 实现阶段
 - 13.2 里程碑

- 13.3 风险与缓解
- 十四、可观测性
 - 14.1 结构化日志
 - 14.2 Metrics 规范
 - 14.3 分布式追踪
 - 14.4 告警规则
 - 14.5 健康检查
- 十五、测试策略与质量保证
 - 15.1 测试分层策略
 - 15.2 单元测试规范
 - 15.3 集成测试方案
 - 15.4 端到端测试
 - 15.5 性能基准
 - 15.6 混沌测试计划
 - 15.7 CI/CD 流水线
- 十六、开发者指南
 - 16.1 SDK 设计
 - 16.2 CLI 工具
 - 16.3 本地开发体验
 - 16.4 调试工具
 - 16.5 错误码参考
- 十七、部署与配置
 - 17.1 系统要求
 - 17.2 本地开发部署
 - 17.3 Podman 单机部署
 - 17.4 配置参考
 - 17.5 网络规划
- 十八、迁移指南
 - 18.1 版本变更摘要
 - 18.2 不兼容变更清单
 - 18.3 渐进迁移路径

- 18.4 环境迁移指南
- 18.5 训练框架迁移指南
- 18.6 回滚策略

一、产品概述

本文档是 UEnv 分布式训练环境框架的产品需求文档（PRD），定义了 UEnv 的产品定位、用户需求和 技术方案。在 LLM 后训练（Post-Training）领域，RLVR、在线强化学习、Agent 训练等场景 都需要与外部环境交互来收集训练信号，但现有生态中每套训练框架都有各自的环境接口协议，导致 环境实现无法跨框架复用、切换训练框架意味着重写全部环境代码。UEnv 通过"训练框架无关"的核 心设计理念解决这一痛点——提供统一的环境接口和协议转换层，让同一套环境实现可以在 ROLL、 VeRL、NeMo-RL、TRL、OpenRLHF 五大主流训练框架间无缝切换。本章包含项目背景、产品定 位、目标用户画像、核心价值主张和术语定义，为后续所有技术设计章节提供统一的语义基础和上下 文理解。

1.1 项目背景

在 LLM 后训练（Post-Training）生态中，RLVR、在线强化学习、Agent 训练等场景都需要与外部 环境交互来收集训练信号。然而，当前生态面临以下核心痛点：

- **重复造轮子** — 每套训练框架（ROLL、VeRL、NeMo-RL、TRL、OpenRLHF）都有各自的环境接口 协议，导致环境实现无法跨框架复用
- **环境不可迁移** — 切换训练框架意味着重写全部环境代码，适配成本极高
- **缺乏统一管理** — 大规模分布式训练缺少统一的环境注册、调度和生命周期管理机制
- **部署碎片化** — 环境从开发到生产的部署路径不清晰，容器化和资源隔离方案不统一

1.2 产品定位

UEnv 是一个**训练框架无关的分布式环境执行框架**，为 LLM 后训练提供统一的 Environment 接 口。它支持 ROLL、VeRL、NeMo-RL、TRL、OpenRLHF 五大主流训练框架，通过 gRPC 双向流实现 Episode 级粒度的高效通信，让同一套环境实现可以在不同训练框架间无缝切换。

1.3 目标用户

角色	需求	UEnv 提供的价值
训练框架开发者	集成多环境支持	通过 Training Adapter 接入，一次集成获得所有环境
环境开发者	一次开发，多框架可用	实现 Environment ABC + 注册，所有训练框架自动可用

RL 训练工程师	配置即用，无需关心部署	环境注册到 UEnvHub，拉取即用，调度自动
平台运维	统一管理和监控训练环境	UEnv Server 提供环境注册/调度/监控一体化管理

1.4 核心价值主张

价值	说明
训练框架无关	通过 Adapter 适配 5 大训练框架（ROLL/VeRL/NeMo-RL/TRL/OpenRLHF）
环境可定制	MCP 工具 + 可组合 Reward（类 nn.Module）+ 可注入执行逻辑
大规模分布式	UEnv Server 调度（注册表 + 负载均衡 + 实例池）+ 多后端 + 自动扩缩容
开发到生产	Process → Podman 渐进部署路径
环境即服务	注册到 UEnvHub，拉取即用；NeMo-Gym 适配器即刻覆盖 43+ 生产级环境

1.5 术语定义

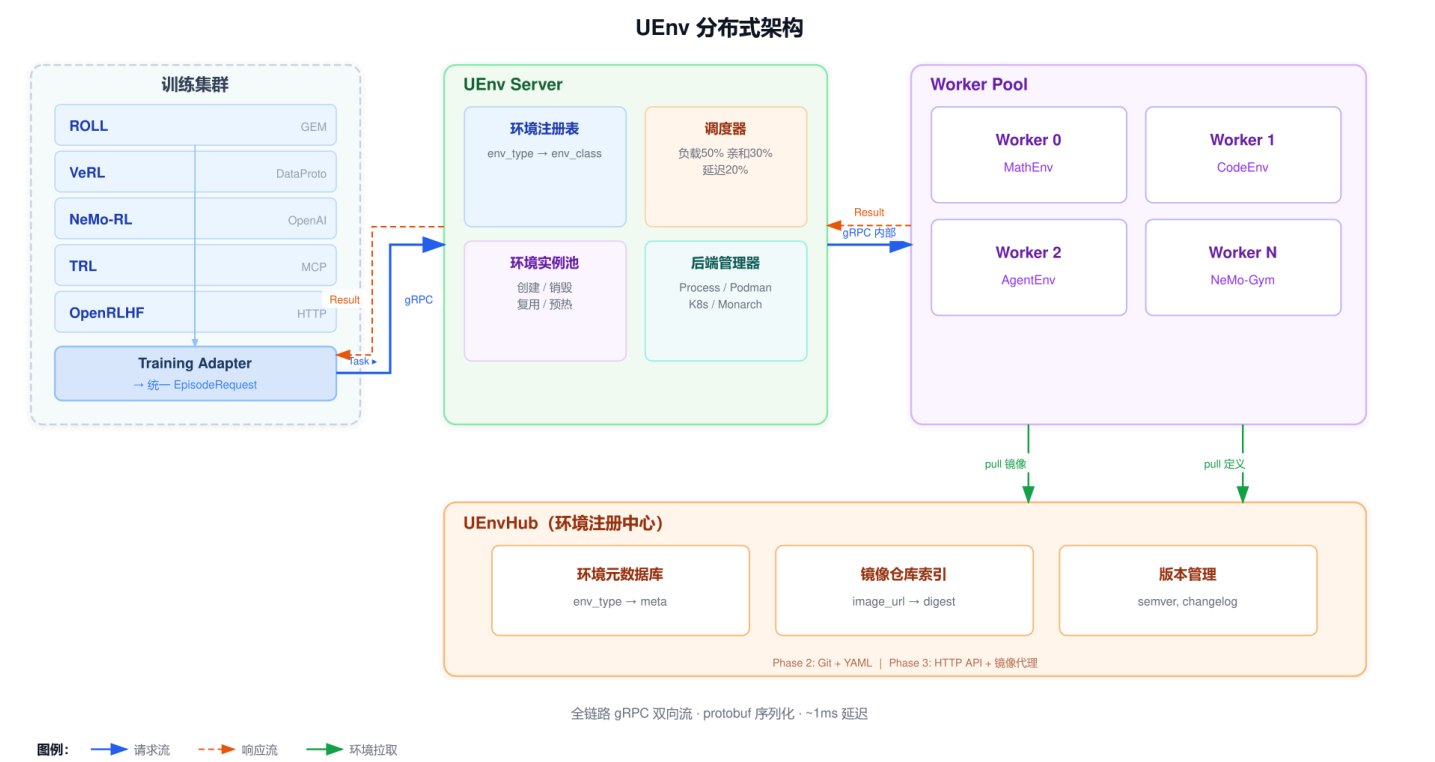
术语	定义
Episode	一次完整的环境交互周期（reset → $N \times \text{step}$ → close）
EpisodeRequest	训练侧发送的环境运行请求（protobuf 消息）
EpisodeResult	环境侧返回的执行结果（protobuf 消息）
Worker	环境执行进程，持有环境实例，执行完整 Episode
UEnv Server	有状态环境管理服务，包含环境注册表、调度器、实例池和后端管理器
UEnvHub	持久化环境注册中心，存储环境元数据和镜像索引（离线目录服务）
Training Adapter	训练框架适配器，将框架协议转换为 UEnv EpisodeRequest
环境实例	Worker 上的一个具体环境对象（创建后可复用，由实例池管理生命周期）

MCP	Model Context Protocol, UEnv 采用其风格设计工具系统
环境注册表	维护 env_type → 环境类 (env_class) 的全局映射

二、系统设计总览

本章是阅读后续所有设计章节的入口。先理解全局架构和分层设计思路，再按需深入各维度章节。

2.1 总体架构



架构说明：

- **左侧（训练集群）**：5 个训练框架通过各自 Adapter 将原生协议转换为统一 EpisodeRequest
- **中间（UEnv Server）**：环境注册表（env_type→env_class 映射）+ 调度器（负载 50% + 亲和 30% + 延迟 20%）+ 实例池（预热/复用）+ 后端管理器
- **右侧（Worker Pool）**：多个 Worker 各持有不同环境实例，执行完整 Episode
- **底部（UEnvHub）**：持久化环境注册中心，Worker 启动时从中拉取环境定义和元数据
- **通信**：全链路 gRPC 双向流，protobuf 序列化，~1ms 延迟

2.2 设计思路

UEnv 的架构遵循**三层解耦 + 控制数据面分离**原则：

第一层解耦：训练框架 ↔ 环境（通过 Adapter + gRPC）

训练框架不直接调用环境，而是通过 Training Adapter 将框架原生协议（GEM/DataProto/OpenAI/MCP）转换为统一的 EpisodeRequest，经 gRPC 发送到 UEnv Server。这意味着：

- 新增训练框架只需开发一个 Adapter，无需修改环境代码
- 环境开发者不需要知道有哪些训练框架在使用它

第二层解耦：调度 ↔ 执行（通过 UEnv Server ↔ Worker）

UEnv Server 负责调度决策（哪个 Worker 有空、资源是否匹配），Worker 负责实际执行 Episode。这意味着：

- 调度逻辑不关心环境内部实现
- Worker 不关心任务从哪个训练框架来
- 调度和执行可以独立扩缩容

第三层解耦：环境定义 ↔ 环境分发（通过 UEnvHub）

环境开发者将环境元数据发布到 UEnvHub（持久化目录），Worker 启动时拉取环境定义。这意味着：

- 环境发布与环境部署是异步的
- UEnvHub 是静态目录，不参与运行时调度
- 环境版本管理与运行时实例管理分离

第四层原则：控制平面与数据平面分离

UEnv Server 是**纯控制平面**——仅负责 Episode 的调度编排、Worker 注册、实例池管理等元数据操作。Episode 执行期间的 step 级数据流（模型推理请求、环境交互、Reward 计算）**不经过 Server**，由 Worker 直接与推理服务（vLLM/SGLang）通信。

代码块

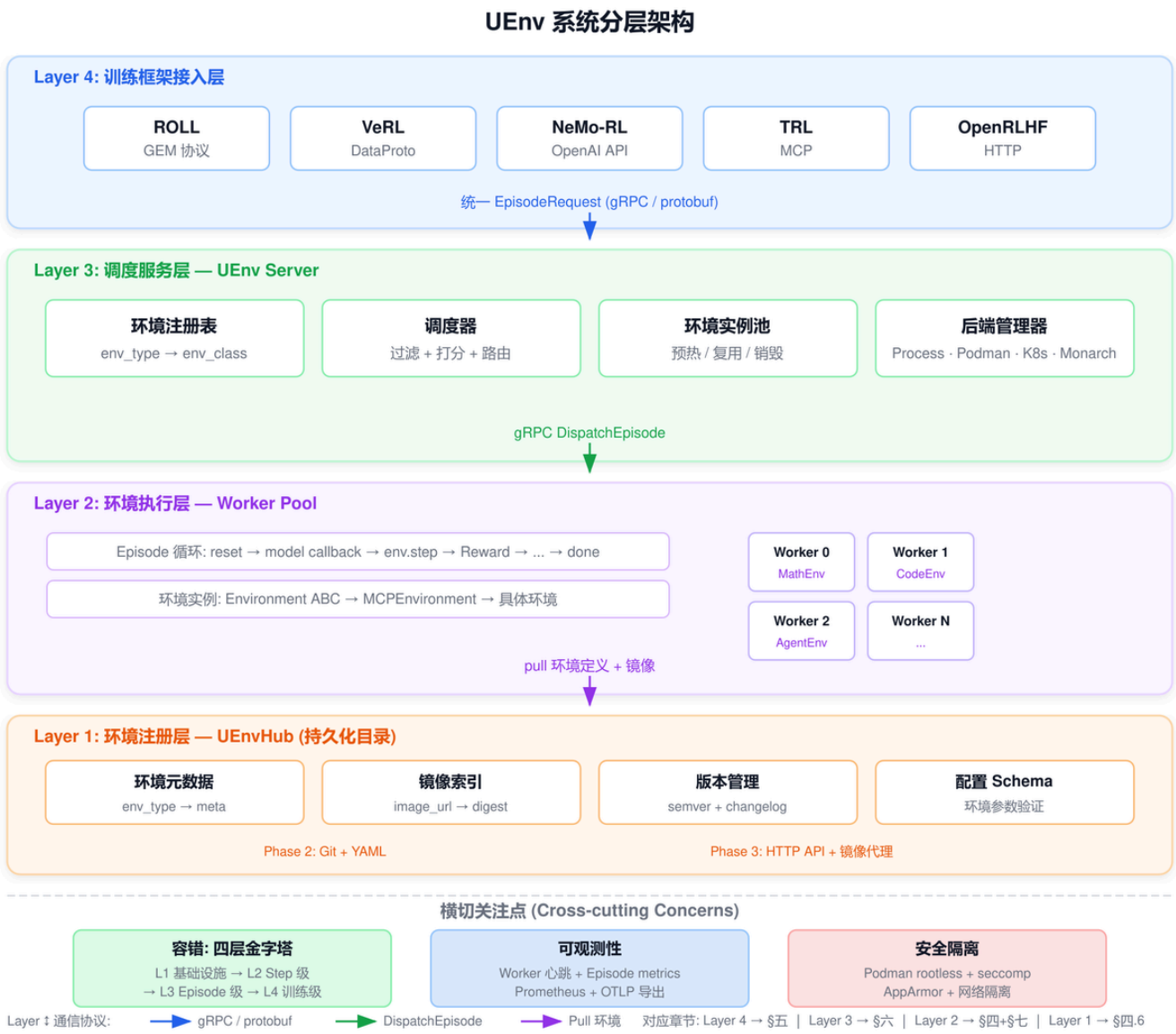
- 1 控制平面（经过 Server）： Episode 提交 → 调度决策 → 分配 Worker → 收集结果
- 2 数据平面（绕过 Server）： Worker → 模型推理（直连） → env.step（本地） → Reward 计算

这一分离确保 Server 不会成为 step 级数据瓶颈，同时保持对 Episode 生命周期的完整编排能力。

适用场景说明： UEnv 的核心定位是 **Agentic RL 的环境框架**（多步交互，5-80 步 Episode）。RLVR 场景（prompt→answer→verify，单步验证）通常通过训练框架内置的 reward model 直接处理，不需要 UEnv 介入。代码库证据：ROLL 的 RLVR Pipeline 使用

AsyncDynamicSamplingScheduler 而非 TrajEnvManager，环境交互路径仅用于 Agent 场景。

2.3 系统分层架构



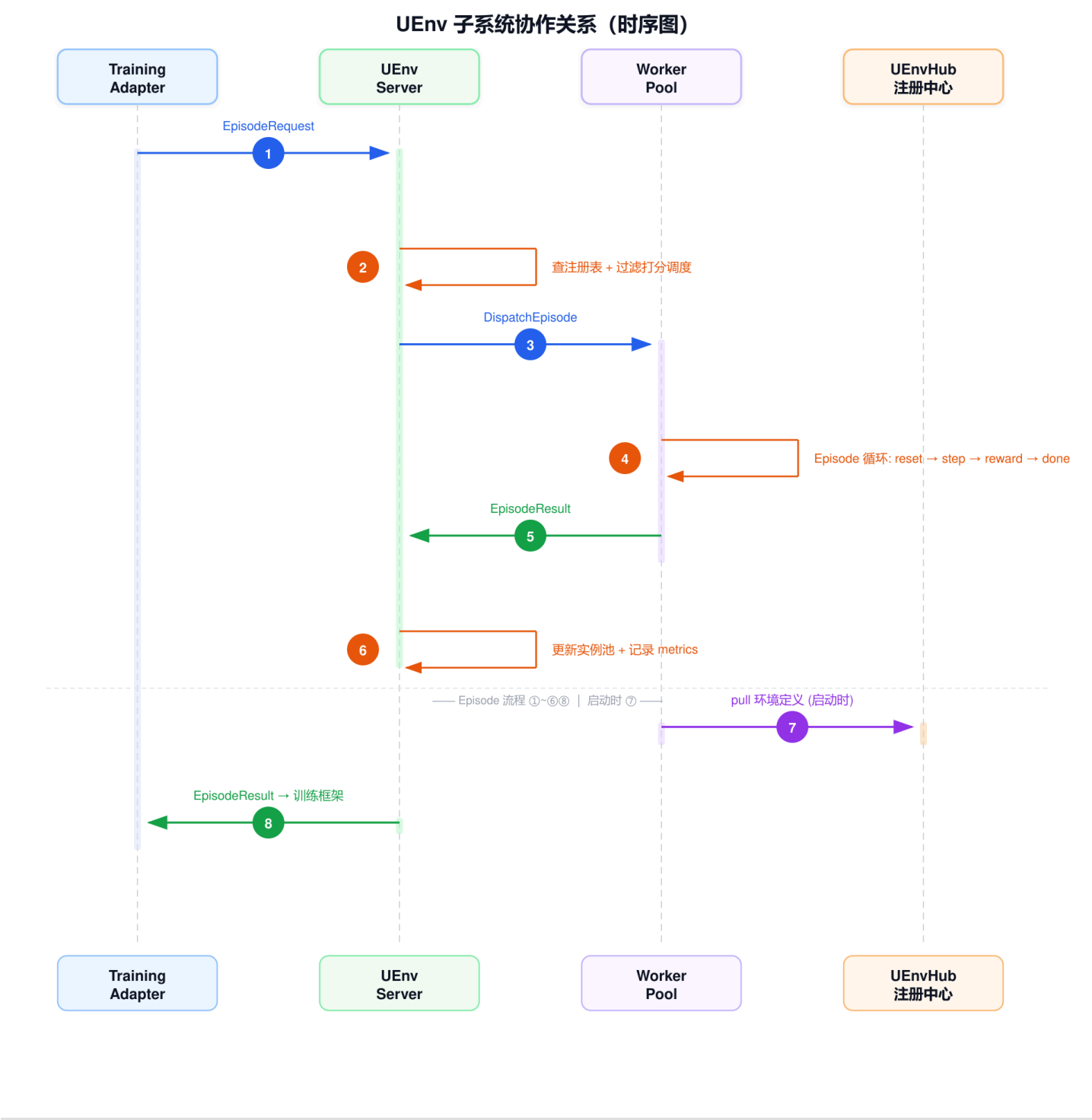
四层职责说明:

层级	核心职责	关键组件	对应设计章节
Layer 4 训练框架接入	协议转换、错误映射、重试	5 个 Training Adapter	§ 五 交互机制
Layer 3 调度服务	请求路由、负载均衡、实例管理、扩缩容	UEnv Server (4 子模块)	§ 六 任务调度

Layer 2 环境执行	Episode 循环、模型回调、Reward 计算	Worker Pool + 环境实例	§ 四 环境构造 + § 七 运行时
Layer 1 环境注册	元数据存储、版本管理、镜像分发	UEnvHub	§ 四.6 环境注册与分发

2.4 子系统协作关系

训练框架提交一个 Episode 到获得结果的完整路径，涉及 5 个子系统的协作：



三、场景与需求

本章描述 UEnv 解决的具体用户场景，以及功能和性能需求。核心思想是"环境开发者写一次，训练工程师用任何框架都能跑"。

3.1 用户故事

故事 1 — 训练工程师使用 ROLL 训练数学模型

训练工程师使用 ROLL 框架训练数学推理模型。通过 GEMAdapter 将 ROLL 的 GEM make/step 调用自动转换为 UEnv EpisodeRequest，UEnv Server 将 Episode 调度到持有 MathEnv 的 Worker，Worker 执行完整 Episode（模型生成答案 → 环境验证 → Reward 计算），返回 trajectory 给 ROLL。工程师无需修改任何 ROLL 代码，只需配置 Adapter endpoint。

故事 2 — 训练工程师使用 VeRL 训练代码生成

训练工程师使用 VeRL 训练代码生成模型。VeRLAdapter 将 DataProto 批量样本转换为并行 EpisodeRequest，通过 gRPC 双向流提交到 UEnv Server，8 个 GPU Worker 并行执行代码沙箱环境，流式回收结果。训练吞吐 ~120 episodes/min。

故事 3 — 训练工程师使用 NeMo-RL 做 Agent 训练

训练工程师使用 NeMo-RL 做 Agent 工具使用训练。NeMoAdapter 将 Ray Actor handle 转换为 HTTP/gRPC 端点信息，AgentEnv 支持多轮 Episode（每步调用模型 → 工具执行 → 状态推进），MCP 工具自动路由 tool_call action 到注册的工具函数，训练侧实时接收 StreamReport 监控进度。

故事 4 — 环境开发者创建新环境

环境开发者需要创建一个安全分析环境。实现 MCPEnvironment 子类，注册 scan_code 和 query_cve 两个 MCP 工具，实现 _step_impl() 方法。通过 @register_env("security") 注册后打包发布到 UEnvHub，所有训练框架立即可用。

故事 5 — 平台运维部署大规模训练集群

平台运维在 Podman 集群上部署 UEnv。配置 PodmanBackend 管理环境容器，预热池根据历史预测提前创建环境实例。Prometheus + Grafana 监控集群状态，Podman rootless + seccomp 确保安全隔离。

3.2 功能需求清单

本节按优先级（P0 到 P3）列出 UEnv 的全部功能需求，每项需求包含编号、功能名称、描述、验收标准和依赖关系。P0 为 MVP 必备功能，确保端到端 Demo 可运行；P1 为核心功能，覆盖双框架支

持和 MCP 工具能力；P2 为生产化功能，包括完整框架覆盖、容错和监控；P3 为高级功能，面向特殊场景和生态扩展。

P0 — MVP 必备

编号	功能	描述
F-01	Protobuf Schema	定义 EpisodeRequest/EpisodeResult/StreamReport 消息
F-02	UEnv Server 核心服务	环境注册表 + 调度器 + 实例池 + gRPC Server
F-03	Worker 基础框架	gRPC Server + Episode 执行循环 + 心跳
F-04	Training Adapter 基类	convert_request/convert_response/execute_episode
F-05	GEMAdapter (ROLL)	GEM make/step/reset/close → EpisodeRequest 转换
F-06	MathEnv	数学问题求解 + 可验证奖励
F-07	ProcessBackend	本地进程后端，零依赖

P1 — 核心功能

编号	功能	描述
F-08	VeRLAdapter	DataProto batch → EpisodeRequest 转换
F-09	NeMoAdapter	BatchedDataDict → EpisodeRequest 转换
F-10	MCPEnvironment	中间层自动路由 list_tools/call_tool/普通 step
F-11	Reward 系统	从 v4.0 迁移，零变更： Sequential/Gate/WeightedSum/RewardList/RewardDict/Mux
F-12	CodeEnv	多语言代码执行 + 沙箱 + 单元测试
F-13	Worker 预热池	提前创建环境实例，Episode 到达时跳过创建延迟
F-14	流式上报	StreamReport 逐步上报进度
F-15	PodmanBackend	Rootless 容器隔离后端

P2 — 生产化

--	--	--

编号	功能	描述
F-16	OpenRLHFAdapter	env.make/step → EpisodeRequest 转换
F-17	TRLAdapter	MCP tool call → EpisodeRequest 转换
F-18	AgentEnv	多轮对话 + 工具调用 + 多智能体协作
F-19	容错 — Worker 级	Worker OOM → 重启 → UEnv Server 重新分配
F-20	容错 — Episode 级	Episode 失败 → 3 次重试
F-21	PodmanBackend	Podman rootless 容器管理 + seccomp 隔离
F-22	UEnvHub Git+YAML	轻量环境元数据仓库
F-23	监控	Prometheus metrics

P3 — 高级功能

编号	功能	描述
F-24	WebEnv	浏览器自动化（Playwright）
F-25	GameEnv	棋盘/物理仿真
F-26	UEnvHub HTTP API	完整环境注册中心（发布/搜索/版本管理）
F-27	环境检查点/恢复	state → snapshot → 恢复
F-28	TrajectoryReward	延迟/轨迹奖励高级信号源

3.4 约束与假设

本节明确 UEnv 设计的技术约束、前提假设和范围排除，帮助读者理解方案的设计边界。同时说明 v6.2 与 v4.0 的关键差异，确保从旧方案迁移时的认知一致性。

技术约束：

- 不使用消息中间件（Redis/Kafka/RabbitMQ），直接 gRPC 通信
- Episode 级粒度调度（非单步），降低通信开销
- gRPC 双向流 + Protobuf 序列化，不使用 JSON
- Reward 系统与 v4.0 完全兼容，零变更

前提假设：

- 训练框架已支持某种协议（GEM/gRPC/OpenAI/MCP），Adapter 可做转换
- 模型推理端点（vLLM/SGLang）已部署并可访问
- Worker 运行环境已安装必要依赖（Python 3.10+ / Podman）

范围排除：

- 不包含训练引擎本身（PPO/GRPO/DAPO 等算法由训练框架负责）
- 不包含推理引擎（vLLM/SGLang 等由模型服务层负责）
- 不包含模型管理和权重分发

四、环境构造方法

本章描述如何从零构建一个可被 UEnv 调度的环境。核心设计思想是"环境不关心谁在调用它"——通过泛型三参数 ABC 定义最小接口，通过 MCP 中间层扩展工具能力，通过可组合 Reward 构建灵活的奖励信号，最终通过四级注册链路将环境代码转化为可调度服务。

4.1 环境定义

Environment 是所有 UEnv 环境的顶层抽象基类，参考 OpenEnv 设计，采用泛型参数 `Environment<ActT, ObsT, StateT>` 约束动作、观测和状态类型。它定义了环境交互的最小接口，所有子类（MCPEnvironment、具体环境）均继承此基类。生命周期遵循 `reset() → N × step() → close()` 的标准模式，与 Gymnasium 兼容。

核心接口契约：

方法	功能	返回值	说明
<code>reset</code>	重置环境到初始状态	<code>ObsT</code>	接受可选 seed 参数，保证可复现性
<code>step</code>	执行一步动作	<code>(ObsT, float, bool, bool, dict)</code>	返回 (观测, 奖励, 是否终止, 是否截断, 附加信息)
<code>close</code>	清理环境资源	<code>None</code>	释放文件句柄、网络连接等
<code>state</code>	获取当前环境状态	<code>StateT</code>	用于检查点/恢复，通过 property 访问

设计原则：

--	--

原则	说明
训练框架无关	环境不关心谁在调用它，只接受 action 返回 observation
Gymnasium 兼容	step 返回五元组 (obs, reward, terminated, truncated, info)，与 Gymnasium 对齐
泛型约束	ActT/ObsT/StateT 三参数约束动作、观测和状态类型
最小接口	reset/step/close/state 四个方法是最小契约，子类可扩展但不缩减

4.2 环境能力扩展

MCPEnvironment 中间层

MCPEnvironment 是 Environment 的中间层子类，为 Agent 训练场景提供**自动工具路由**能力。子类只需实现 `_step_impl()` 方法定义环境特定的 step 逻辑，工具调用通过 MCP 协议自动分发到已注册的工具函数。

自动路由机制：

当 `step(action)` 收到一个 action 字典时，MCPEnvironment 根据 `action["type"]` 字段自动判断并路由：

action 类型	路由目标	处理逻辑
"list_tools"	内部处理	遍历已注册工具，返回名称、描述和参数 Schema 列表
"call_tool"	注册的工具函数	根据 <code>action["tool_name"]</code> 查找工具，调用 <code>tool.execute(action["arguments"])</code> 返回结果
其他	<code>_step_impl()</code>	委托给子类实现的普通 step 逻辑

工具注册与执行流程：

代码块

```
1  注册阶段：
2    环境开发者 → @tool_register("bash", ...) → MCPTool 实例 → register_tool() →
   _tools dict
3
4  执行阶段：
5    Agent 发送 {"type": "call_tool", "tool_name": "bash", "arguments": {...}}
6      → MCPEnvironment.step() 自动路由
7      → _tools["bash"].execute(arguments) → 结果返回给 Agent
```

双 API 边界：

- **Simulation API**（训练侧）：Worker 内部调用 env.reset/step/close，训练循环自动管理
- **Production API**（推理侧）：通过 MCP 工具调用协议与 Agent 交互，支持 list_tools/call_tool

4.3 奖励系统

UEnv 完全复用 v4.0 的类 `nn.Module` 可组合奖励系统，零变更。奖励系统分为四层：信号源层产生原始奖励信号，容器层组合多个信号，信用分配层将 Episode 级奖励分配到步骤级，归一化层标准化奖励分布。

奖励系统架构：

代码块

```
1  Reward 基类 (类 nn.Module)
2  |—— forward(ctx: RewardContext) → float | Tensor
3  |—— __setattr__ 自动注册子 Reward
4  |—— hooks: register_forward_hook / register_forward_pre_hook
5  |—— state_dict() / load_state_dict()
6
7  容器层 (可组合) :
8  |—— Sequential(*rewards)          # 顺序门控, 零值快速失败
9  |—— Gate(reward, threshold)       # 阈值包装
10 |—— WeightedSum(rewards, weights) # 加权组合, async 并行
11 |—— RewardList / RewardDict      # 动态容器
12 |—— Mux(selector, reward_dict)   # 多任务路由
13
14 信号源层:
15 |—— RuleReward(func)              # 规则奖励 (验证器)
16 |—— NeuralRewardModel(model)     # 神经网络 RM
17 |—— LLMJudge(prompt, client)     # LLM 评判 (async)
18 |—— PRIMEReward(policy, ref)     # 隐式 PRM (token 级)
19 |—— TrajectoryReward(...)        # 延迟/轨迹奖励
20
21 信用分配层:
22 |—— ExponentialDiscounting(gamma) # 指数折扣
23 |—— UniformCredit()              # 均匀分配
24 |—— TokenLevelPRM(granularity)   # token 级信用
25
26 归一化层:
27 |—— NormalizeThenSum()           # 先各自归一化再加权
28 |—— SumThenNormalize()          # 先加权求和再归一化 advantage
29 |—— GroupNorm()                 # 组内 z-score 归一化
```


信号源性能特征：

信号源	类型	延迟	成本
RuleReward	同步规则	<10ms	零
NeuralRewardModel	神经网络推理	~100ms	GPU
LLMJudge	异步 LLM 调用	~2s	API
PRIMEReward	隐式策略对比	~50ms	GPU
TrajectoryReward	延迟计算	变化	视实现

组合示例（代码生成奖励链）：

为代码生成任务构建多级奖励链：第一步，通过 Gate + RuleReward 确保代码必须编译通过；第二步，通过 Gate + RuleReward 确保至少通过一半测试；第三步，通过 WeightedSum 加权评估测试通过率（50%）+ 神经网络 RM 评分（20%）+ LLM 评判代码质量（30%）。任何一步失败（零值），Sequential 立即短路返回零。

4.4 环境执行模式

Worker 内部的 Episode 执行引擎是 UEnv 的运行时核心。它负责从接收 EpisodeRequest 到返回 EpisodeResult 的完整流程。

执行步骤：

1. 获取环境实例：从预热池获取或新建环境实例
2. 初始化 Reward 链：根据 RewardConfig 构建可组合的 Reward 树
3. 环境 Reset：调用 `env.reset(seed)` 初始化环境状态，记录初始观测
4. 主循环（每步迭代，最多 `max_steps` 步）：
 - 4a. 超时检查：若当前时间超过 deadline，立即返回 TIMEOUT + partial trajectory
 - 4b. 模型回调：将观测和轨迹历史发送到 model_endpoint，获取 action 和 logprob
 - 4c. 环境 Step：调用 `env.step(action)`，若是 MCP 工具调用则自动路由
 - 4d. Reward 计算：构建 RewardContext，通过 Reward 链计算综合奖励
 - 4e. 记录：将本步的观测/动作/奖励/终止标志等写入 Trajectory
 - 4f. 流式上报：通过 StreamReport 向 UEnv Server 上报 STEP_COMPLETE
 - 4g. 终止检查：若 terminated 或 truncated 为 True，跳出循环
5. TrajectoryReward：计算跨步骤的延迟奖励（如折扣累计、整体质量评估）

6. **汇总返回**：构建 EpisodeSummary（总奖励、步数、延迟、终止原因），连同 Trajectory 封装为 EpisodeResult 返回
7. **实例归还**：将环境实例归还预热池（而非销毁），供后续 Episode 复用

四种执行模式：

模式	说明	适用场景
单轮	一步完成，action = 完整输出	数学、问答、代码生成
多轮	Agent 多次 step，每步调用模型	工具使用、多步推理
模型回调	Worker 内部循环调用模型	Agent 训练、ReAct 循环
可定制	用户自定义执行循环	复杂环境逻辑

模型回调机制：

Worker 内部通过 model_endpoint（HTTP/gRPC/Ray Actor）调用 vLLM/SGLang 等推理服务，获取模型输出作为 action。每次调用携带观测和轨迹历史，模型返回 action 及其 logprob，用于 PPO 等算法的策略梯度计算。

4.5 环境容器化

Podman 隔离执行

Worker 在 Podman 容器中的典型部署配置：

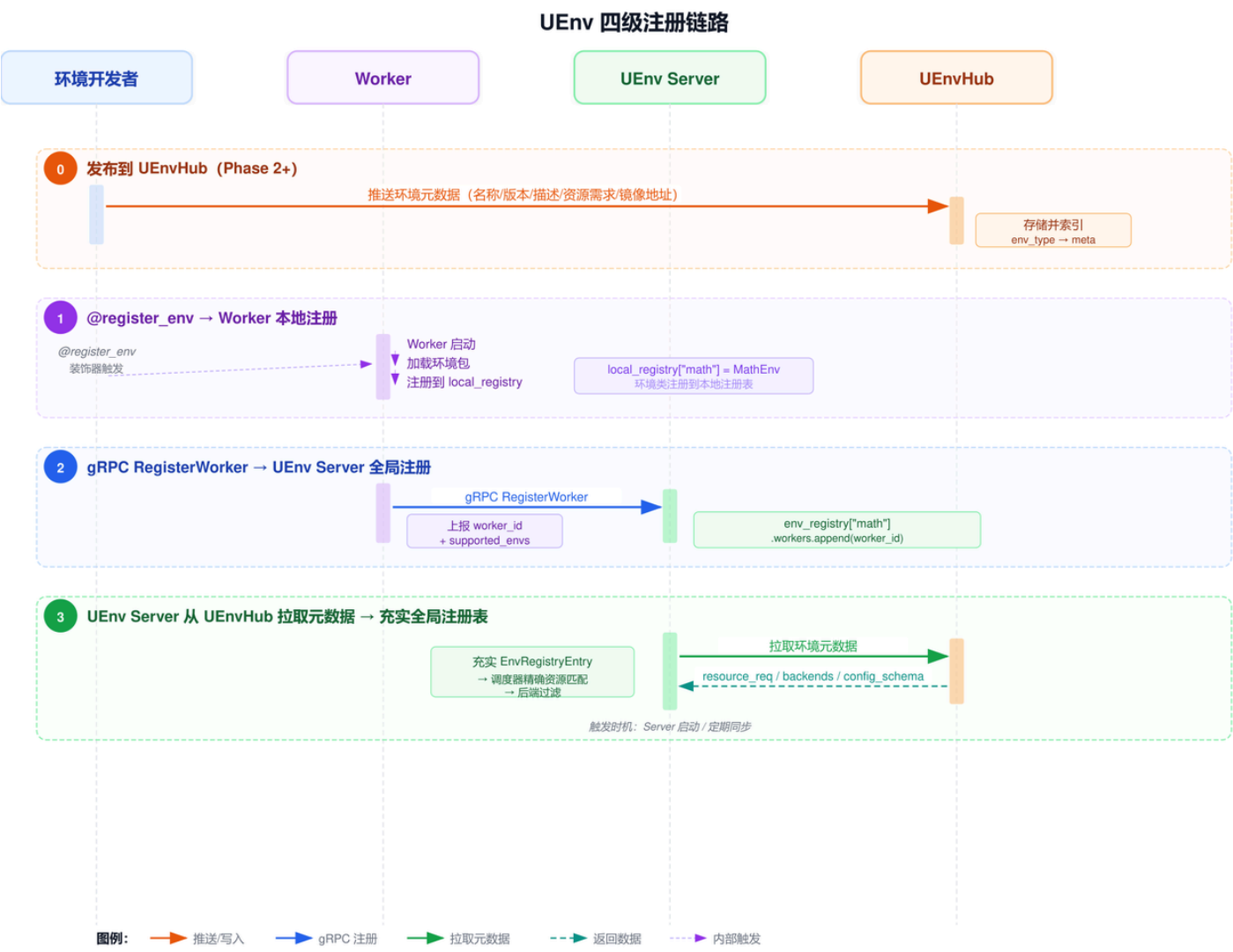
配置项	值	说明
Worker 镜像	uenv/worker:v5.0	包含环境执行引擎和基础环境
环境类型	UENV_ENV_TYPES=math,math_proof	声明该 Worker 支持的环境
Server 连接	UENV_SERVER_ENDPOINT=dns:///uenv-server:50051	UEnv Server gRPC 地址
最大并发	UENV_MAX_CONCURRENT_EPISODES=8	每 Worker 并发 Episode 上限
CPU 限制	4 核	资源配额
内存限制	8G	资源配额
健康检查	gRPC 健康检查，每 10s	Worker 崩溃时自动恢复

安全策略：

安全机制	说明
Podman Rootless	无需 root 权限，用户命名空间隔离
Seccomp	自定义系统调用白名单，限制危险 syscall
AppArmor	容器级强制访问控制
资源限制	CPU/内存/GPU 约束，防止资源逃逸

4.6 环境注册与分发

环境注册是 UEnv 将环境能力从"代码实现"转化为"可调度服务"的关键过程。



四级注册链路：

代码块

1 第零级：发布到 UEnvHub (Phase 2+)

```
2  环境开发者 → 推送环境元数据（名称/版本/描述/资源需求/镜像地址） → UEnvHub 存储并索引
3
4  第一级：开发者 @register_env → Worker 本地注册
5  Worker 启动 → 加载环境包 → 环境类注册到本地注册表 local_registry["math"] =
  MathEnv
6
7  第二级：Worker 启动 → gRPC RegisterWorker → UEnv Server 全局注册
8  Worker 通过 gRPC RegisterWorker 上报 worker_id + supported_envs
9  UEnv Server 更新全局注册表 env_registry["math"].workers.append(worker_id)
10
11 第三级：UEnv Server 从 UEnvHub 拉取环境元数据 → 充实全局注册表
12  UEnv Server 启动/定期同步 → 拉取 resource_requirements, supported_backends,
  config_schema
13  充实 EnvRegistryEntry → 调度器可做精确资源匹配和后端过滤
```

注册职责划分：

注册级别	执行者	职责	存储位置
UEnvHub 发布	环境开发者	发布环境元数据到 UEnvHub	UEnvHub（Git/HTTP）
开发者注册	环境开发者	@register_env 声明环境类型名和资源需求	Python 包内
Worker 本地注册	Worker 启动流程	加载环境包，将环境类实例化到本地注册表	Worker 内存
Server 全局注册	UEnv Server	Worker 上报 + UEnvHub 拉取元数据，更新全局路由表	UEnv Server 内存

UEnvHub 环境注册中心：

UEnvHub 是 UEnv 的持久化环境注册中心，不是运行时组件，而是离线目录服务。

维度	UEnvHub（持久化目录）	UEnv Server 全局注册表（运行时状态）
存储内容	环境类型、版本、镜像地址、资源需求、配置 Schema	在线 Worker 列表、Worker 持有的环境实例

生命周期	永久（环境发布后一直存在）	临时（Worker 下线后清除）
类比	Podman Hub	Podman daemon 的 container list

UEnvHub 三阶段演进：

阶段	实现方式	功能	适用期
Phase 2	Git 仓库 + YAML manifest	Worker git pull 拉取环境列表	2-3 个内置环境
Phase 3	HTTP API 服务 + 对象存储	环境发布/搜索/版本管理/镜像代理	多团队协作
Phase 4	生态化（Web UI + 评分 + 自动测试）	环境市场、使用量统计、社区评价	开源生态

环境 Manifest Schema：

字段	类型	必填	说明
env_type	string	✓	环境类型标识（全局唯一）
version	string	✓	语义化版本（semver）
description	string	✓	环境功能描述
author	string	✓	作者/团队
image	string	⚠	容器镜像地址（Podman 后端必填）
entrypoint	string	⚠	入口命令（Process 后端使用）
supported_backends	list	✓	支持的后端列表
resource_requirements	object	✓	资源需求（CPU/内存/GPU）
config_schema	object	✗	环境配置 JSON Schema
dependencies	list	✗	依赖的其他环境
tags	list	✗	搜索标签

UEnvHub 在调度中的作用：

- 1. 资源匹配：根据 resource_requirements 确保只有足够 CPU/GPU 的 Worker 才能接收该环境的 Episode
- 2. 后端过滤：根据 supported_backends 过滤掉不支持该环境后端的 Worker
- 3. 配置验证：根据 config_schema 验证 EpisodeRequest 中的 env_config 是否合法
- 4. 版本管理：同一 env_type 可以有多个版本，EpisodeRequest 可以指定版本

4.7 内置环境策略

"3 核心 + NeMo-Gym 适配器"策略

UEnv 采用"3 种核心自研 + 适配器扩展"策略。数学、代码、Agent 三类环境覆盖了 80% 以上的后训练场景，而游戏、Web、工具等长尾场景通过 NeMo-Gym 适配器快速接入。这样既保证核心环境的质量和深度优化，又通过适配器实现生态覆盖的广度。

环境矩阵表：

环境	类型	核心功能	适用训练场景	对标参考	实现阶段
MathEnv	验证型	数学问题求解、Lean 定理证明、可验证奖励	RLVR 数学、形式化验证	NeMo-Gym Math, GSM8K, MATH	Phase 0
CodeEnv	执行型	多语言代码执行（Python/C++/Java）、沙箱安全、单元测试	代码生成、程序修复	NeMo-Gym Code, HumanEval, MBPP	Phase 0
AgentEnv	交互型	多轮对话、工具调用、多智能体协作	Agent 训练、工具学习	AEnvironment, AutoGPT, SWE-Agent	Phase 1
GameEnv	仿真型	棋盘游戏、物理仿真、策略游戏	策略 RL、多智能体	Gymnasium, PettingZoo, Unity ML	Phase 4
WebEnv	自动化型	浏览器控制、表单填写、网页抓取	Web Agent、自动化	BrowserGym, WebArena, Mind2Web	Phase 4
ToolEnv	工具型	Shell 执行、API 调用、文件操作、数	工具学习、代码解释器	OpenAI Code Interpreter, SWE-	Phase 4

		数据库		Agent	
--	--	-----	--	-------	--

NeMo-Gym 适配器覆盖：

NeMo-Gym 适配器将 43+ NeMo-Gym 环境桥接到 UEnv 统一接口，实现零成本生态复用。适配器内部将 NeMo-Gym 的观测格式转换为 UEnv 统一 dict 格式，将 UEnv 的 action 转换为 NeMo-Gym 原生格式。

领域	UEnv 核心自研	NeMo-Gym 覆盖	合计覆盖
数学	MathEnv（代数/几何/微积分/Lean证明）	6 个（GSM8K, MATH, Lean, Minerva, MathQA, DeepMind）	7
代码	CodeEnv（Python/C++/Java/沙箱）	3 个（HumanEval, MBPP, APPS）	4
Agent	AgentEnv（多轮对话/工具调用）	5 个（Aviary, ToolQA, WebShop, AlfWorld, ScienceWorld）	6
科学	—	1 个（ScienceAgent）	1
工具	—	2 个（ToolAgent, GoogleSearch）	2

五、交互机制

本章描述各组件之间如何通信、协议是什么。核心设计思想是"gRPC 双向流 + 有状态 UEnv Server"——直接 RPC 通信避免消息中间件的额外延迟和运维成本，双向流原生支持长 Episode 的中间状态上报。

5.1 通信架构选型

三种方案对比：

--	--	--	--	--

维度	gRPC 双向流	Ray Actor	HTTP REST
延迟	★★★★ ~1ms	★★ ~5ms	★ ~10ms
流式支持	✓ 原生双向流	⚠ 需要 Ray Serve	✗ 需要 SSE/WebSocket
类型安全	✓ .proto 强类型	⚠ Pickle 序列化	✗ JSON 弱类型
语言支持	✓ 全语言	✗ Python only	✓ 全语言
中间状态上报	✓ 双向流	⚠ 需额外机制	⚠ 需要 SSE
序列化性能	★★★★ protobuf 5-10x	★★ Pickle	★ JSON
水平扩展	✓ UEnv Server 调度	⚠ Ray Cluster	✓ 负载均衡

选型结论：gRPC 双向流 + 有状态 UEnv Server

核心理由：

- 1. 行业事实标准 — NeMo-RL、VeRL、OpenRLHF 均使用直接 RPC，无消息中间件
- 2. 流式支持 — gRPC 双向流原生支持长 Episode 的中间状态上报
- 3. 性能 — protobuf 比 JSON 序列化快 5-10x，trajectory 数据量大时优势明显
- 4. 统一调度 — UEnv Server 内置注册表 + 调度器 + 实例池，提供环境级调度决策

为什么不用消息中间件：

问题	影响
额外运维成本	Redis/Kafka 集群管理、监控、升级
延迟增加	每条消息经过 broker，增加 2-5ms
序列化开销	消息中间件强制 JSON，无法用 protobuf
流式不支持	中间状态上报需要额外机制
主流框架不用	无一个主流训练框架使用消息队列

5.2 通信协议

本节定义 UEnv 系统中四个核心 Protobuf 消息类型的完整字段规格。EpisodeRequest 是训练侧发起的环境运行请求，EpisodeResult 是环境执行完毕返回的结果，StreamReport 用于执行过程中的实时进度反馈，EpisodeError 描述各类错误的详情和处理策略。这些消息构成了训练框架与环境之间数据交换的契约。

EpisodeRequest

EpisodeRequest 是训练侧发送给 UEnv Server 的核心请求消息。

字段	类型	说明
身份标识		
request_id	string	全局唯一 ID（UUID v4）
correlation_id	string	关联 ID，匹配 Response
framework	string	训练框架标识： "roll"/"verl"/"nemo_rl"/"openrlhf"/"trl"
request_ts	double	请求时间（Unix epoch）
环境规格		
env_type	string	环境类型： "math"/"code"/"agent"/"game"/"web"/"tool"
env_config	bytes	JSON 编码的环境配置
env_id	string	复用已有环境实例（可选）
模型端点		
model_endpoint.endpoint_type	string	"http"/"grpc"/"ray_actor"
model_endpoint.url	string	推理服务 URL
model_endpoint.model_name	string	多模型场景的模型名
model_endpoint.api_key	string	API 密钥
model_endpoint.headers	bytes	额外 HTTP 头（JSON）
model_endpoint.generation_config	bytes	生成配置（temperature/max_tokens/stop 等，JSON）

model_endpoint.max_retries	int32	推理最大重试次数（默认 3）
Episode 配置		
episode_config.max_steps	int32	最大步数（默认 100）
episode_config.max_turns	int32	最大对话轮次（默认 10）
episode_config.seed	int32	随机种子
episode_config.initial_observation	bytes	初始观察（JSON）
episode_config.stop_conditions	string[]	停止条件列表
episode_config.system_prompt	string	系统提示词
episode_config.extra_messages	bytes	额外消息上下文（JSON）
奖励配置		
reward_config.reward_type	string	"rubric"/"external"/"hybrid"
reward_config.rubric_config	bytes	Sequential/Gate/WeightedSum 配置（JSON）
reward_config.external_endpoint	string	外部奖励端点 URL
reward_config.reward_shaping	bytes	gamma/clip 等配置（JSON）
reward_config.component_weights	bytes	奖励组件权重（JSON）
元数据		
metadata	bytes	框架自定义元数据（JSON）
priority	int32	0=普通, 1=高, 2=紧急
timeout_seconds	double	超时（默认 300s）
response_endpoint	string	直连返回的 gRPC endpoint

EpisodeResult

EpisodeResult 是 Worker 执行完 Episode 后返回的结果消息。

字段	类型	说明
身份标识		
request_id	string	请求 ID
correlation_id	string	关联 ID
worker_id	string	执行 Worker ID
response_ts	double	响应时间
状态		
status	enum	COMPLETED / FAILED / PARTIAL / TIMEOUT
error	EpisodeError	失败时的错误详情
轨迹数据		
trajectory.steps	StepRecord[]	每步记录列表
trajectory.total_steps	int32	总步数
trajectory.total_tokens	int32	总 token 数
trajectory.initial_observation	bytes	初始观测
trajectory.final_observation	bytes	最终观测
StepRecord 字段		
step_index	int32	步序号
observation	bytes	本步观测（JSON）
action	string	执行的动作
action_logprob	double	动作对数概率
reward	double	累计奖励

step_reward	double	本步奖励
terminated	bool	是否终止
truncated	bool	是否截断
info	bytes	附加信息（JSON）
req_tokens	int32	请求 token 数
resp_tokens	int32	响应 token 数
latency_ms	double	本步延迟
汇总		
summary.total_reward	double	总奖励
summary.reward_components	bytes	各奖励组件分值（JSON）
summary.episode_length	int32	Episode 长度
summary.total_model_calls	int32	模型调用次数
summary.total_latency_ms	double	总延迟
summary.model_latency_ms	double	模型推理延迟
summary.env_latency_ms	double	环境执行延迟
summary.success	bool	是否成功
summary.termination_reason	string	终止原因： "done"/"max_steps"/"timeout"/"error"
数据完整性		
trajectory_checksum	bytes	trajectory 的 SHA-256 校验和
integrity_verified	bool	Worker 端已校验

StreamReport

StreamReport 用于 Episode 执行过程中的实时进度反馈。

字段	类型	说明
request_id	string	请求 ID
worker_id	string	Worker ID
report_type	enum	PROGRESS / STEP_COMPLETE / REWARD_SIGNAL / LOG
current_step	int32	当前步数
total_steps	int32	总步数
payload	bytes	类型相关数据（JSON）
estimated_remaining_seconds	double	预估剩余时间（仅 PROGRESS 类型）

EpisodeError

字段	类型	说明
error_type	string	"env_error" / "model_error" / "timeout" / "reward_error" / "internal" / "config_error"
error_message	string	错误详情
error_step	int32	出错步骤
partial_trajectory	Trajectory	已完成部分

错误处理策略：

错误类型	处理方式	是否重试
env_error	返回 partial_trajectory + error	Adapter 决定
model_error		✅ max_retries 次

	返回 error，由 Adapter 重试推理	
timeout	返回 partial_trajectory	✗
reward_error	返回 partial_trajectory（缺 reward）	✗
internal	Worker 重启，返回 error	UEnv Server 重新分配
config_error	立即返回 error	✗

5.3 gRPC Service 设计

UEnv 定义了三组 gRPC Service，各有明确的职责边界：

UEnvService — 面向训练侧和 Worker：

方法	请求 → 响应	说明
SubmitEpisode	EpisodeRequest → EpisodeResult	训练侧提交单个 Episode
SubmitEpisodeStream	stream EpisodeRequest → stream EpisodeResult	批量提交，流式返回
DispatchEpisode	EpisodeRequest → stream StreamReport	UEnv Server 分发到 Worker
HealthCheck	HealthCheckRequest → HealthCheckResponse	健康检查
ManageEnvironment	ManageRequest → ManageResponse	预创建/销毁环境、Worker 排空

WorkerDirectService — Worker 主动上报：

方法	请求 → 响应	说明
ReportResult	EpisodeResult → Ack	Worker 返回 Episode 执行结果

DispatcherService — Worker 注册与心跳：

方法	请求 → 响应	说明
RegisterWorker	WorkerInfo → RegistrationResponse	Worker 启动时注册
WorkerHeartbeat	stream HeartbeatRequest → stream HeartbeatResponse	Worker 心跳保活（双向流）

5.4 Training Adapter 层

通用 Adapter 职责：

Training Adapter 是训练框架与 UEnv 之间的协议转换层，核心职责有三：

- 1. 请求转换：框架原生格式 → EpisodeRequest （ convert_request ）
- 2. 响应转换：EpisodeResult → 框架原生格式 （ convert_response ）
- 3. 执行编排：组合转换与提交为一个完整流程 （ execute_episode / execute_batch ）

通用接口签名：

方法	功能
convert_request(framework_request) → EpisodeRequest	将框架原生请求转换为标准 EpisodeRequest
convert_response(EpisodeResult) → framework_response	将 EpisodeResult 转回框架原生格式
execute_episode(framework_request) → framework_response	完整流程：转换 → 提交 → 转换返回
execute_batch(requests) → AsyncIterator	批量提交，流式返回结果

五个框架 Adapter 对比：

--	--	--	--	--

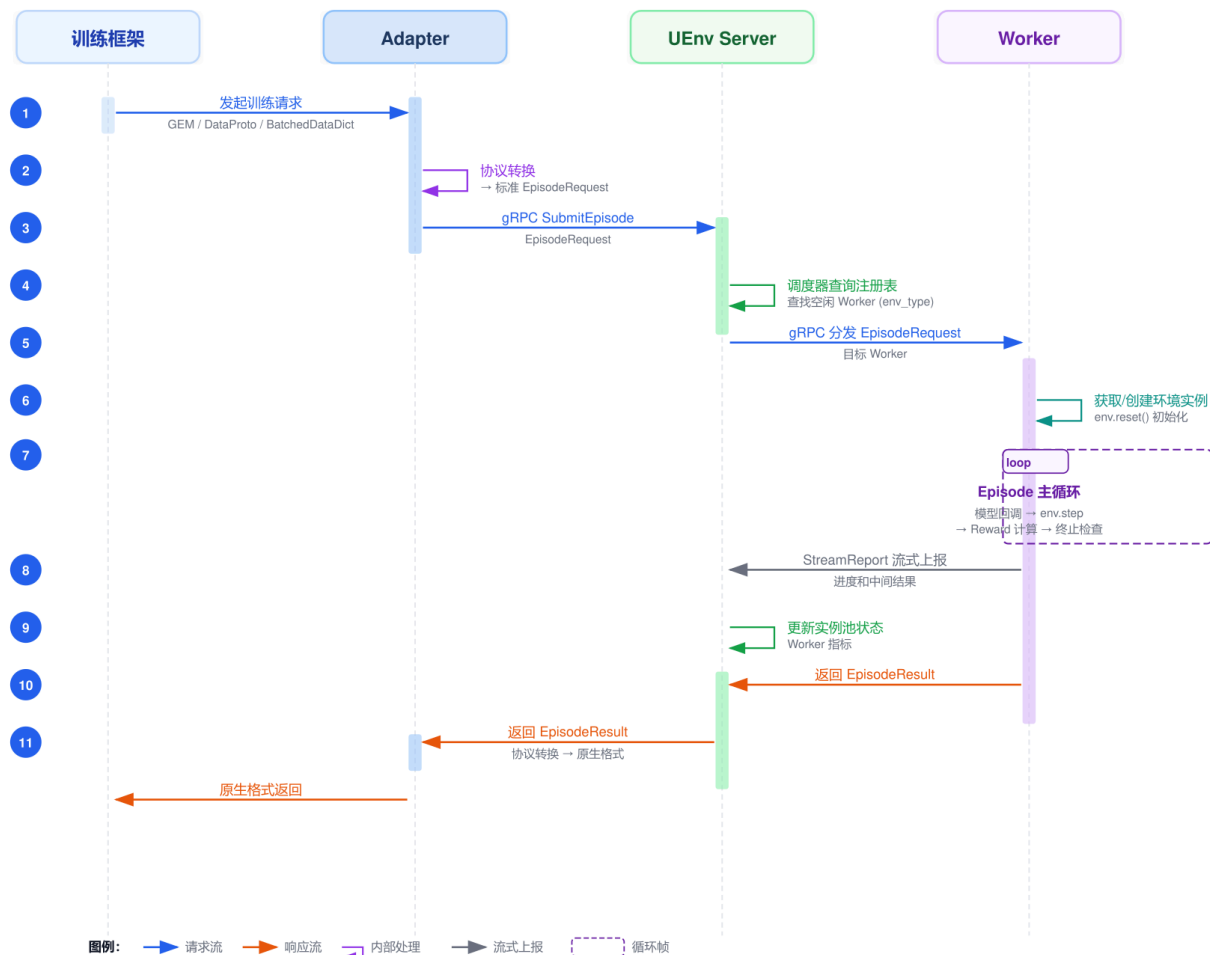
Adapter	框架	输入协议	输出格式	集成难度
GEMAdapter	ROLL	GEM make/step/reset/close → EpisodeRequest	GEM result dict (observations/reward s/dones/total_reward)	低（GEM 协议清晰）
VeRLAdapter	VeRL	DataProto batch → EpisodeRequest (task_name → env_type)	DataProto 格式轨迹数 据	中（DataProto 结构较 复杂）
NeMoAdapter	NeMo-RL	BatchedDataDict + Ray Actor handle → EpisodeRequest	BatchedDataDict 格式 结果	中（Ray Actor 转端点）
OpenRLHFAdap ter	OpenRL HF	env.make/step 序列 → EpisodeRequest	OpenRLHF 兼容 step 返 回格式	低
TRLAdapter	TRL	MCP tool call context → EpisodeRequest (env_type="tool")	MCP tool call 响应格式	低

部署模式：

模式	说明	优点	缺点
嵌入式 （推荐）	Adapter 部署在训练集 群，靠近训练框架	低延迟，独立扩缩	需要在训练侧部署
内嵌式	Adapter 路由集成在 UEnv Server 内	简单部署，单进程	UEnv Server 负担重，单 点风险
Sidecar	Adapter 作为 Sidecar 与 训练 Worker 同 Pod	网络优化	运维复杂度高

5.5 Episode 完整交互时序

Episode 完整交互时序



各阶段说明:

1. 训练框架发起训练请求 (GEM task dict / DataProto / BatchedDataDict 等)
2. Adapter 将框架原生格式转换为标准 EpisodeRequest
3. Adapter 通过 gRPC 调用 UEnv Server 的 SubmitEpisode 接口
4. UEnv Server 调度器查询环境注册表, 查找支持该 env_type 的空闲 Worker
5. UEnv Server 通过 gRPC 将 EpisodeRequest 分发到目标 Worker
6. Worker 获取/创建环境实例, 调用 env.reset() 初始化
7. Worker 执行 Episode 主循环 (每步: 模型回调 → env.step → Reward 计算 → 终止检查)
8. Worker 通过 StreamReport 流式上报进度和中间结果
9. UEnv Server 更新实例池状态和 Worker 指标
10. Episode 完成后, Worker 返回 EpisodeResult 给 UEnv Server
11. UEnv Server 将 EpisodeResult 返回给 Adapter, Adapter 转换为框架原生格式返回

5.6 流控与背压机制

SubmitEpisodeStream 批量提交场景下，需要防止 Server 内存被排队请求撑爆。

gRPC 流控：

代码块

```
1  # Adapter 端：控制发送速率
2  async def submit_batch(stub, requests):
3      channel = grpc.aio.insecure_channel(SERVER_ENDPOINT)
4      # 设置发送窗口
5      await channel._channel.check_connectivity_state(True)
6
7      call = stub.SubmitEpisodeStream()
8      for req in requests:
9          # 检查对端是否准备好接收
10         if not call.is_active():
11             break
12         await call.write(req)
13     call.done_writing()
14
15     async for result in call:
16         yield result
```

Server 端背压：

代码块

```
1  # Server 端：控制读取速率
2  async def SubmitEpisodeStream(self, request_iterator, context):
3      queue = asyncio.Queue(maxsize=1000) # 有界队列
4
5      async def producer():
6          async for req in request_iterator:
7              if queue.full():
8                  # 发送 FlowControlMessage 通知暂停
9                  await context.write(FlowControlMessage(
10                      action=PAUSE,
11                      reason="queue_full"
12                  ))
13                  await queue.put(req) # 阻塞等待
14              else:
15                  await queue.put(req)
16
```

FlowControlMessage 定义:

代码块

```

1  message FlowControlMessage {
2      enum Action {
3          PAUSE = 0;    // 暂停发送
4          RESUME = 1;   // 恢复发送
5          THROTTLE = 2; // 限速发送
6      }
7      Action action = 1;
8      string reason = 2;
9      int32 suggested_batch_size = 3; // THROTTLE 时的建议批次大小
10 }

```

5.7 消息大小与传输策略

Agent 训练的多轮 Episode trajectory 可能超过 gRPC 默认 4MB 限制。

消息大小配置:

参数	值	说明
max_message_size	64 MB	gRPC channel 级别配置
max_receive_message_size	64 MB	Server 端接收限制
max_send_message_size	64 MB	Client 端发送限制
compression	gzip	大消息自动启用

大 Trajectory 处理策略:

1. **自动压缩:** Trajectory 序列化后 > 1MB 时自动 gzip 压缩, 在 metadata 中标记 `content-encoding: gzip`

2. **分片传输**（v7.1 规划）：超过 64MB 的 trajectory 拆分为多个 chunk，附带 `chunk_index` 和 `total_chunks`
3. **截断策略**：保留最近 N 步（可配置），丢弃早期 step 的 observation/detail，仅保留 summary

gRPC Channel 配置示例：

代码块

```
1 channel_options = [  
2     ('grpc.max_send_message_length', 64 * 1024 * 1024),  
3     ('grpc.max_receive_message_length', 64 * 1024 * 1024),  
4     ('grpc.default_compression_algorithm',  
5     grpc.CompressionAlgorithm.gzip),  
6 ]
```

5.8 AdminService

运维管理专用 gRPC Service，支持查询、排空、取消等管理操作。

代码块

```
1 service AdminService {  
2     // Worker 管理  
3     rpc ListWorkers(ListWorkersRequest) returns (ListWorkersResponse);  
4     rpc GetWorker(GetWorkerRequest) returns (WorkerDetail);  
5     rpc DrainWorker(DrainWorkerRequest) returns (DrainWorkerResponse);  
6  
7     // 环境实例管理  
8     rpc ListInstances(ListInstancesRequest) returns (ListInstancesResponse);  
9     rpc GetInstance(GetInstanceRequest) returns (InstanceDetail);  
10  
11    // Episode 管理  
12    rpc GetEpisodeStatus(GetEpisodeStatusRequest) returns (EpisodeStatus);  
13    rpc CancelEpisode(CancelEpisodeRequest) returns (CancelEpisodeResponse);  
14  
15    // 系统状态  
16    rpc GetServerStatus(GetServerStatusRequest) returns (ServerStatus);  
17 }  
18  
19 message ListWorkersRequest {
```

```

20     string env_type_filter = 1; // 可选: 按环境类型过滤
21     string status_filter = 2;   // 可选: 按状态过滤
22 }
23
24 message ListWorkersResponse {
25     repeated WorkerSummary workers = 1;
26     int32 total_count = 2;
27 }
28
29 message WorkerDetail {
30     string worker_id = 1;
31     string status = 2;
32     repeated string supported_env_types = 3;
33     int32 active_episodes = 4;
34     int32 total_episodes_completed = 5;
35     float avg_episode_duration_seconds = 6;
36     int64 registered_at = 7;      // Unix timestamp
37     int64 last_heartbeat_at = 8;
38 }
39
40 message DrainWorkerRequest {
41     string worker_id = 1;
42     int32 timeout_seconds = 2;    // 排空超时, 超时后强制终止
43     bool force = 3;              // 立即终止, 不等待进行中 Episode 完成
44 }
45
46 message CancelEpisodeRequest {
47     string episode_id = 1;
48     string reason = 2;
49 }
50
51 message ServerStatus {
52     string version = 1;
53     string protocol_version = 2;
54     int32 total_workers = 3;
55     int32 active_workers = 4;
56     int32 pending_episodes = 5;
57     int32 in_flight_episodes = 6;
58     int64 uptime_seconds = 7;
59     float scheduler_queue_utilization = 8;
60 }

```

访问控制: AdminService 要求 `admin` 角色 (详见 § 14.2 RBAC) 。

5.9 批量提交语义 (SubmitBatch)

GRPO/PPO 等训练算法需要批量收集 Episode 结果后才能更新策略。当前

`SubmitEpisodeStream` 是流式返回 (Episode 完成一个返回一个)，缺少 batch completion 语义。新增 `SubmitBatch` RPC 解决同步屏障问题：

代码块

```
1  service UEnvService {
2      // ... 已有方法
3
4      // 批量提交：等待整批完成后统一返回
5      rpc SubmitBatch(BatchRequest) returns (BatchResult);
6  }
7
8  message BatchRequest {
9      string batch_id = 1;
10     int32 batch_size = 2;
11     repeated EpisodeRequest episodes = 3;
12     int32 timeout_seconds = 4;           // 整批超时
13     bool partial_results = 5;           // 超时后是否返回已完成的部分结果
14 }
15
16 message BatchResult {
17     string batch_id = 1;
18     int32 completed_count = 2;
19     int32 failed_count = 3;
20     int32 total_count = 4;
21     bool batch_complete = 5;           // 是否整批完成
22     repeated EpisodeResult results = 6;
23 }
```

与 ROLL 的 `GroupQueueManager` 对齐：ROLL 的 `GroupQueueManager` 收集每个 `group_id` 的 `group_size` 个 rollout 后释放给训练循环。`SubmitBatch` 在 UEnv 层面实现相同的 batch 聚合语义。

5.10 异步提交模式 (SubmitEpisodeAsync)

高并发场景 (>500 eps/s) 下，同步阻塞模式限制推理 GPU 利用率。异步模式允许 Adapter 提交后立即返回，不阻塞训练循环：

代码块

```
1  service UEnvService {
2      // ... 已有方法
3
4      // 异步提交: 立即返回确认
5      rpc SubmitEpisodeAsync(EpisodeRequest) returns (SubmitAck);
6
7      // 查询结果: 按 episode_id 查询
8      rpc GetEpisodeResult(GetResultRequest) returns (EpisodeResult);
9
10     // 结果推送: 服务端主动推送已完成的结果
11     rpc WatchEpisodes(WatchRequest) returns (stream EpisodeResult);
12 }
13
14 message SubmitAck {
15     string episode_id = 1;
16     bool accepted = 2;
17     int32 queue_position = 3;           // 排队位置
18     int32 estimated_wait_seconds = 4; // 预估等待时间
19 }
```

5.11 通信模式对比

UEnv 支持三种通信模式，适配不同训练场景和性能需求：

模式	RPC 方法	适用场景	推理 GPU 利用率
同步	SubmitEpisode	调试、单 Episode、简单训练	~25%（请求串行到达）
流式	SubmitEpisodeStream	批量训练（当前 ROLL 模式）	~50%（请求交织）
批量	SubmitBatch	GRPO/PPO 需要整批完成的场景	~50%
异步	SubmitEpisodeAsync + WatchEpisodes	高并发（>500 eps/s）	~90%（推理请求最大化并发）

同步 vs 异步效率分析：

代码块 同步模式 (Phase 0) :

```
2 Worker 线程1: —推理—等待—结果—推理—等待—结果—
3 Worker 线程2: —推理—等待—结果—推理—等待—结果—
4 → 每线程同时仅 1 个推理请求在途 → GPU batch 小
5
6 异步模式 (Phase 2) :
7 Worker 协程1: —发推理—发推理—发推理—收结果—收结果—
8 Worker 协程2: —发推理—发推理—发推理—收结果—收结果—
9 → 所有协程推理请求汇聚 → GPU continuous batching → batch 大
10 → 推理 GPU 利用率从 ~25% 提升到 ~90%
```

Worker 端推荐的混合模型 (Phase 2) :

代码块

```
1 async def execute_episode(request: EpisodeRequest):
2     """Worker 端异步 Episode 执行"""
3     env.reset(request)
4     while not terminated:
5         # 异步推理调用 (不阻塞其他协程)
6         llm_response = await model_client.generate(...)
7         # 同步 env.step() 在线程池中执行
8         obs, reward = await run_in_executor(env.step, action)
9     return result
```

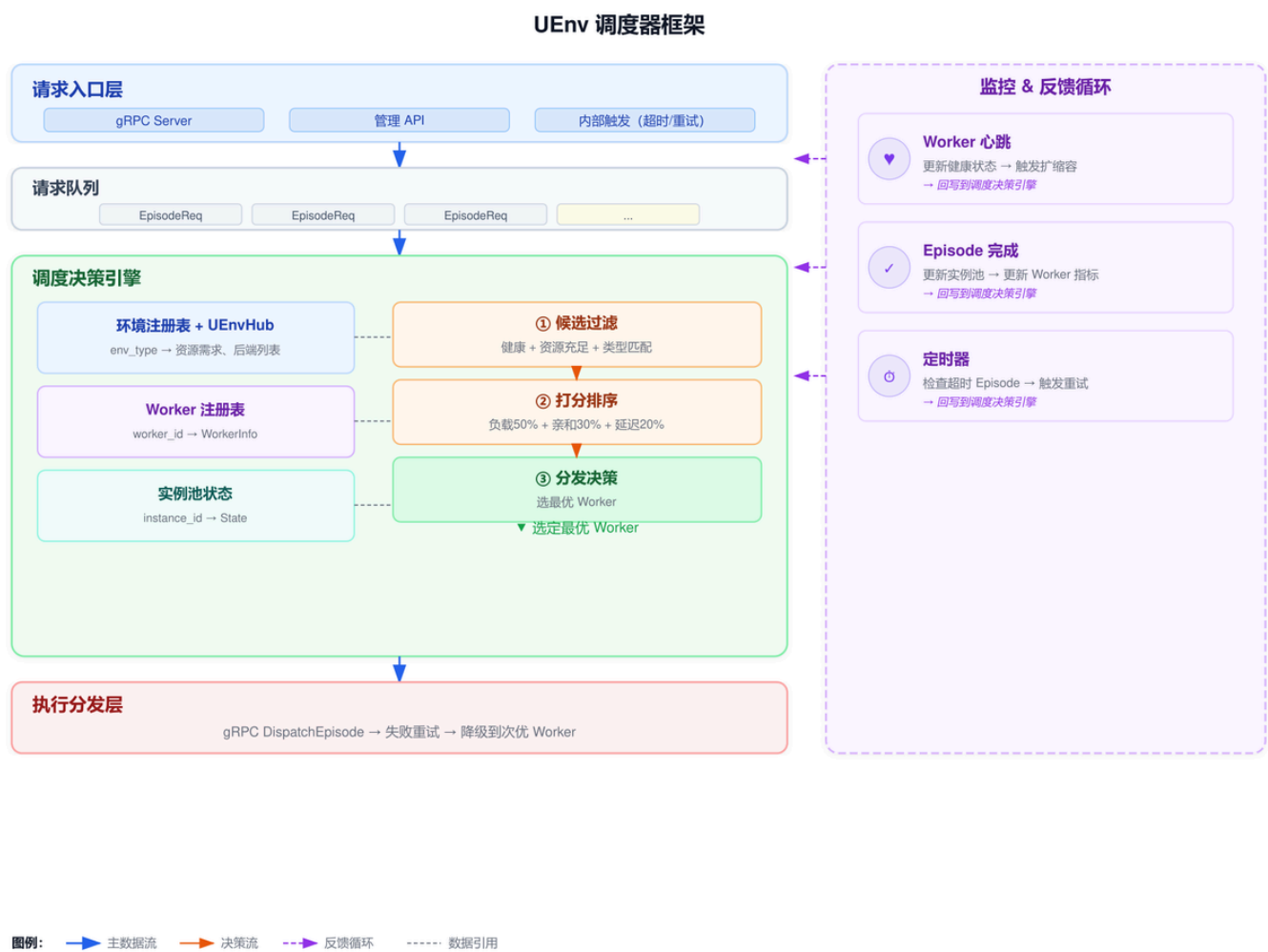
每层使用最适合的模式：Server 内部异步 (asyncio)、Worker 推理调用异步、环境执行同步包装、Adapter 端流式或批量。

六、任务调度

本章描述 Episode 如何被接收、路由、分发、监控的完整调度链。核心设计思想是"**Server 即控制平面**"——UEnv Server 仅负责调度编排 (哪个 Worker 执行、何时执行、如何分配)，不参与 Episode 执行期间的 step 级数据流。Worker 在执行期间直接与推理服务通信 (详见 § 2.2 控制数据面分离原则)。

6.1 调度器框架

UEnv 调度器采用**四层架构**设计：

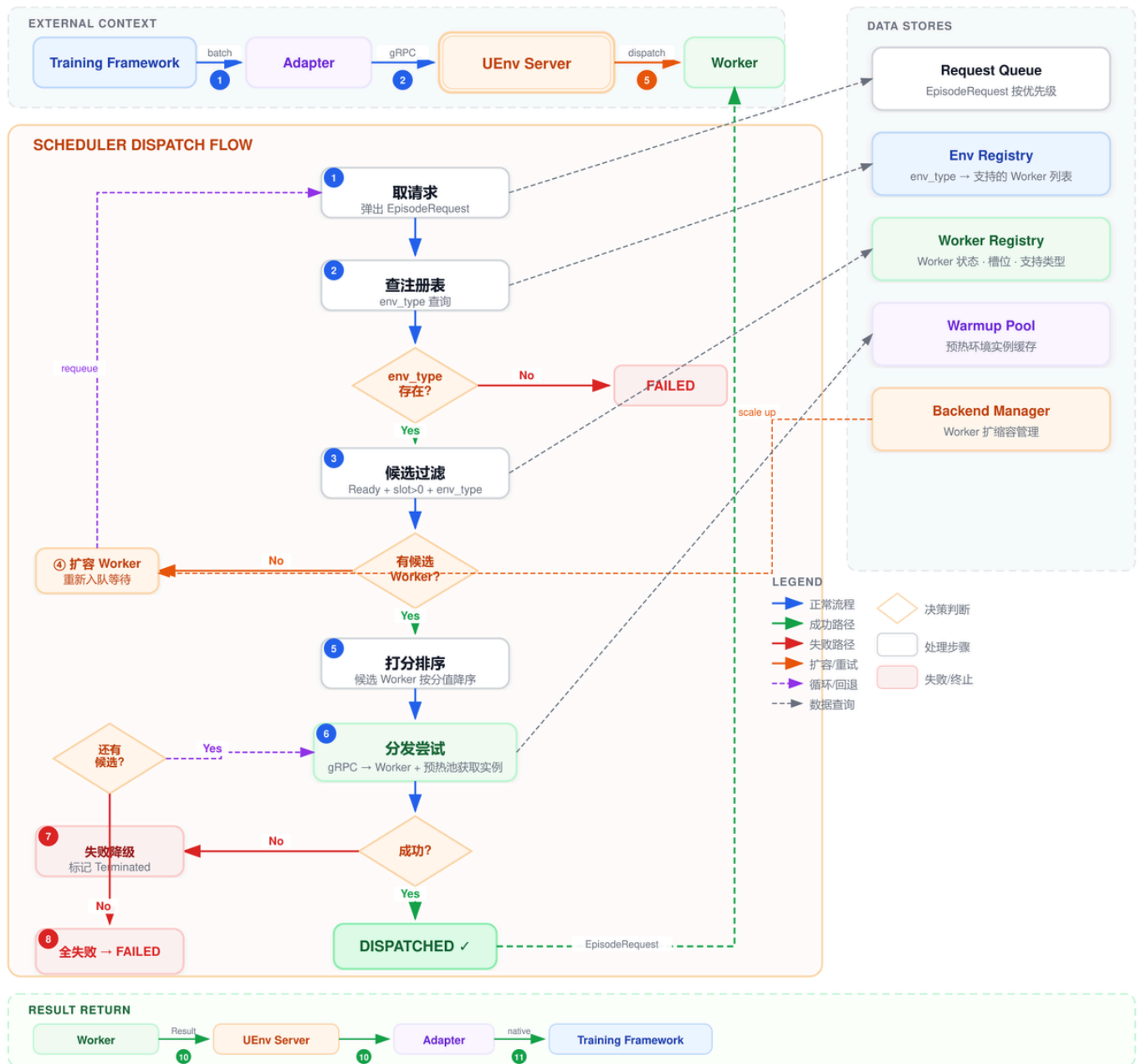


当 Worker 启动并注册其支持的环境类型时，UEnv Server 会从 UEnvHub 拉取该环境类型的元数据（资源需求、兼容后端列表），用于更精确的调度决策——例如确保 GPU 环境只分配到有 GPU 的 Worker 节点。

6.2 调度流程

调度流程 — UEnv Scheduler Dispatch

从 SubmitEpisode 到完成分发, 经历 8 步调度决策



调度器从接收 EpisodeRequest 到完成分发, 经历以下 8 步:

取请求: 从请求队列中弹出下一个 EpisodeRequest (按优先级排序)

2. 查注册表: 根据 env_type 查询环境注册表, 若不存在则直接返回 FAILED

3. 候选过滤: 从 Worker 注册表中筛选——支持该 env_type、状态为 Ready、可用槽位 > 0

4. 无候选处理: 若无可用 Worker, 尝试通过后端管理器扩容创建新 Worker, 将请求重新入队等待

5. 打分排序: 对每个候选 Worker 计算 $\text{load_score} \times 0.5 + \text{affinity_score} \times 0.3 + \text{latency_score} \times 0.2$, 按总分降序排列

6. 分发尝试: 从最高分 Worker 开始, 尝试从预热池获取环境实例, 通过 gRPC 转发 EpisodeRequest

7. 失败降级: 若当前 Worker 不可用 (抛出 WorkerUnavailable), 标记其为 Terminated, 尝试下一个候选

8. 全失败兜底：若所有候选 Worker 均不可用，返回 FAILED 错误

打分算法：

得分维度	计算方式	权重	设计意图
负载得分	$\frac{\text{available_slots}}{\text{max_concurrent_episodes}}$	50%	优先选择负载较轻的 Worker
亲和性得分	已持有同类型环境 = 1.0，否则 = 0.3	30%	避免冷启动，复用已有环境实例
延迟得分	$1.0 - \min(\frac{\text{avg_model_latency}}{5000\text{ms}}, 1.0)$	20%	优先选择历史延迟低的 Worker

关键决策点：

决策点	条件	动作
环境不存在	注册表中无该 env_type	直接返回 FAILED，不重试
无可用 Worker	候选列表为空	尝试扩容或排队等待
Worker 不可用	gRPC 分发抛出异常	标记 Terminated，尝试次优
全部失败	所有候选均不可用	返回 FAILED 错误

6.3 调度策略

本节汇总 UEnv 支持的四种调度策略及其适用场景。默认采用"最少负载"策略，确保 Episode 均匀分布；在需要避免环境冷启动时切换为"类型亲和"策略；GPU 等特殊资源场景使用"资源感知"策略；多租户混合优先级任务使用"优先级"抢占策略。

策略	说明	适用场景

最少负载	选择 in-flight episodes 最少的 Worker	默认策略，通用场景
类型亲和	优先选择已持有同类型环境的 Worker	避免冷启动，环境预热
资源感知	根据环境资源需求匹配 Worker 节点	GPU 环境，特殊资源需求
优先级	高优先级 Task 抢占式调度	多租户，混合优先级任务

6.4 环境预热池

预热池是降低环境冷启动延迟的关键机制。调度器根据历史请求模式，提前在 Worker 上创建环境实例并保持在 Warm 状态，当 EpisodeRequest 到达时可直接从池中获取，跳过环境创建（~100ms-2s）的等待。

预热机制：

- 根据历史请求模式预测环境类型需求，提前创建实例
- LRU 保留最近 N 个环境类型的实例
- Episode 完成后实例归还预热池而非销毁

配置参数：

参数	默认值	说明
warmup_pool_size	5	预热池大小
max_idle_time	300s	最大空闲时间，超时后回收
cool_timeout	60s	冷却期超时
max_episode_count	1000	实例最大复用次数（防止内存泄漏）

6.5 扩缩容策略

本节定义 UEnv 各组件的自动扩缩容触发条件和动作。Worker Pod 基于 KEDA 自定义指标（活跃 Episode 数与容量比值）自动扩缩，UEnv Server 基于 CPU 利用率 HPA 扩容，预热池基于历史预测提前创建实例。缩容采用空闲超时策略，避免资源浪费。

组件	扩缩策略	触发条件	动作
UEnv Server	HPA（CPU > 70%）	请求排队延迟 > 100ms	自动扩容
Worker Pod	KEDA（自定义指标）	活跃 Episode 数 / 容量 > 80%	自动创建新 Worker
GPU Worker	手动 / Prometheus Alert	GPU 利用率持续 > 90%	人工扩容
预热池	基于历史预测	训练 Job 启动前	提前预热环境实例
缩容	空闲超时	Worker 空闲超过 idle_timeout	自动销毁释放资源

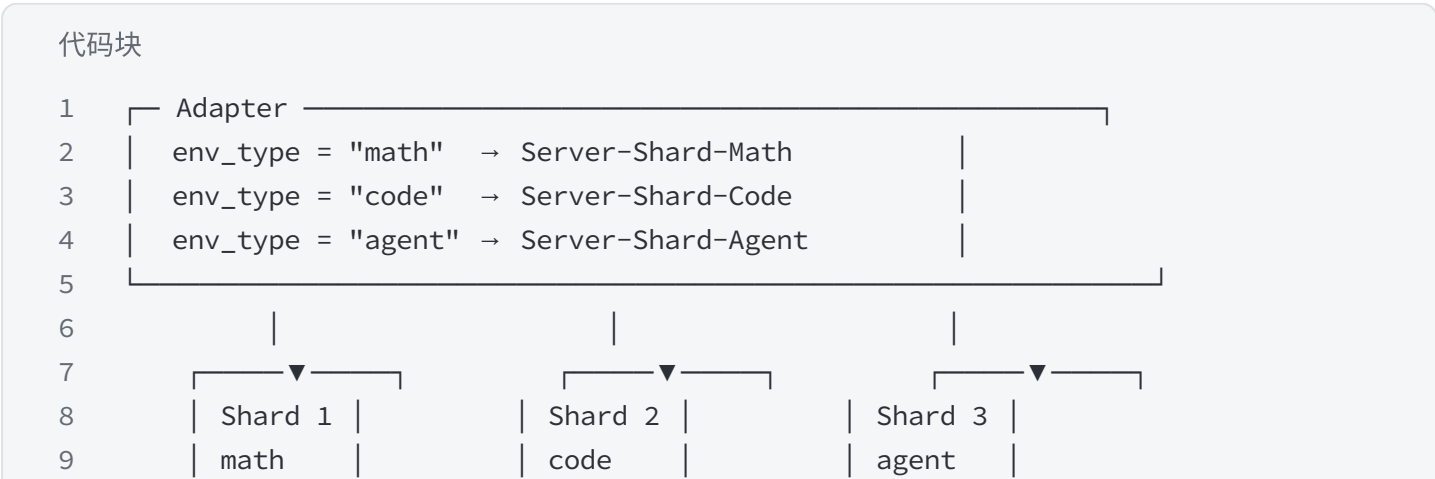
自动扩容逻辑：

- 监控指标：pending_tasks（排队中） / active_workers（活跃 Worker）
- 扩容条件： $\text{pending_tasks} > \text{active_workers} \times 2 \rightarrow$ 通知后端创建新 Worker
- 缩容条件：Worker 空闲超过 idle_timeout $\rightarrow$ 销毁

6.6 UEnv Server 水平扩展

单 UEnv Server 在大规模场景下（100+ Worker、1000+ eps/s）成为瓶颈。v7.0 引入按环境类型分片的水平扩展方案。

分片策略：





分片路由：

- Adapter 端维护 `env_type → Server endpoint` 路由表
- 路由表通过 UEnvHub 获取
- 跨分片 Episode 不支持（设计约束）：每个 `env_type` 的 Worker 只注册到对应分片

分片元数据：

代码块

```
1 shards:
2   - env_types: ["math", "math_verification"]
3     server_endpoint: "uenv-server-math:50051"
4     max_workers: 200
5   - env_types: ["code", "code_execution"]
6     server_endpoint: "uenv-server-code:50051"
7     max_workers: 100
```

6.7 自适应心跳

大规模集群下固定心跳间隔会导致 Server 心跳处理负载过高。

自适应间隔公式：

代码块

```
1 heartbeat_interval = base_interval × min(√(N / 10), 3)
```

其中 `base_interval = 5s`，`N` = 已注册 Worker 数量。

Worker 数量	心跳间隔	Server 每秒心跳处理
10	5.0 s	2/s
50	11.2 s	4.5/s

100	15.8 s	6.3/s
500	35.4 s	14.1/s
1000	47.4 s	21.1/s

实现：

- Server 在每次 WorkerHeartbeat 响应中返回 `next_heartbeat_interval_ms`
- Worker 按照返回的间隔调整下次心跳
- Server 端超时阈值 = `3 × 心跳间隔` （容忍 2 次丢失）

6.8 Server 性能建模

单 UEnv Server 的资源消耗可建模如下：

调度性能：

代码块

1

调度吞吐 = 1 / (过滤耗时 + 打分耗时 + 分配耗时)

2

$\approx 1 / (0.1\text{ms} + 0.5\text{ms} + 0.1\text{ms}) \approx 1400 \text{ ops/s}$

内存模型：

组件	单条大小	100 Worker × 1000 eps/s
WorkerInfo 注册表	~2 KB	200 KB
环境实例池	~1 KB	~1 MB (1000 实例)
进行中 Episode 跟踪	~0.5 KB	~500 KB (1000 in-flight)
调度队列	~1 KB	~10 MB (队列深度 10000)
总计	—	~12 MB

CPU 模型：

- 调度决策：~0.7ms/episode → 1000 eps/s ≈ 0.7 核
- 心跳处理：~0.01ms/次 → 100 Worker × 10s 间隔 ≈ 10/s ≈ 忽略
- gRPC 序列化/反序列化：~0.1ms/episode → 0.1 核
- **推荐配置**：2 核 4GB（100 Worker 场景），4 核 8GB（500+ Worker 场景）

6.9 分阶段状态管理策略

UEnv Server 的状态管理随规模渐进演进，避免过早引入复杂性：

Phase	Worker 规模	状态方案	RTO	理由
Phase 0	<10	进程内状态 + WAL	<5s	最简实现，WAL 保证崩溃恢复
Phase 1	10-50	进程内状态 + WAL + Active-Passive	<30s	主备切换，备用 Server 从 WAL 恢复
Phase 2	50-200	状态外部化到 etcd/Redis	<10s	Server 无状态化，可随时重启/水平扩展
Phase 3	200+	嵌入式共识（etcd 内嵌）+ 分片	<5s	Raft 共识保证一致性，分片水平扩展

状态外部化架构（Phase 2）：

代码块

```
1  UEnv Server (无状态)
2    | 读写调度状态
3    ▼
4  etcd / Redis
5    ├── worker_registry    ← Worker 注册表
6    ├── instance_pool      ← 环境实例池
7    ├── scheduler_queue    ← 调度队列
8    └── episode_tracker     ← 进行中 Episode 追踪
```

Server 可随时重启/替换，从外部存储恢复全部状态。多个 Server 实例可通过 leader election 实现主备切换。

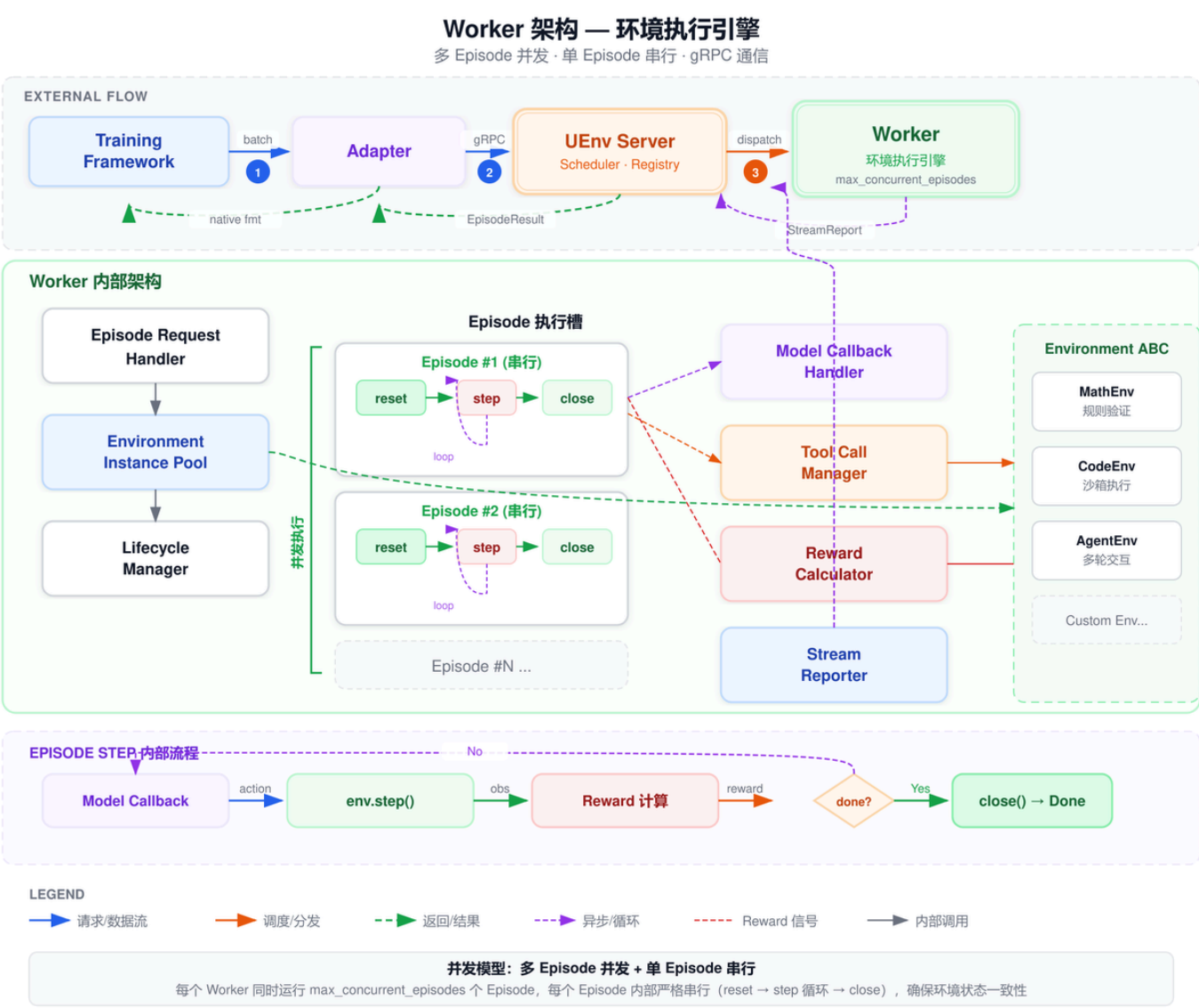
锁竞争缓解：调度器采用分区锁策略——每个 env_type 的调度队列独立加锁，避免全局锁竞争。

七、运行时管理

本章描述环境实例在哪里运行、Worker 如何管理、后端如何选择。核心设计思想是"Backend 对 Worker 透明"——Worker 只调用统一的 Backend trait 方法，不感知底层是进程还是 Podman 容器，实现"开发→集成→生产"的渐进部署路径。

7.1 Worker 架构

Worker 是环境执行引擎，是实际运行 Environment 实例的工作节点。



核心职责:

- 接收 UEnv Server 分发的 EpisodeRequest, 执行完整 Episode
- 内嵌环境实例 (Environment ABC), 管理环境生命周期

- 管理工具调用、模型回调、Reward 计算
- Episode 完成后通过 UEnv Server 返回 EpisodeResult

Worker 内部并发模型：

Worker 采用"多 Episode 并发 + 单 Episode 串行"的并发架构。每个 Worker 可同时运行多个 Episode（由 max_concurrent_episodes 控制），每个 Episode 内部严格串行执行（reset → step 循环 → close），确保环境状态的一致性。

并发维度	模型	说明
Episode 间	并发（异步任务池）	多个 Episode 在同一 Worker 内并行执行，共享预热池
Episode 内	串行	reset → N × step → close 严格顺序，保证环境状态一致性
模型回调	异步非阻塞	通过 HTTP/gRPC 调用推理服务时，当前 Episode 让出执行权，等待响应时处理其他 Episode
工具调用	异步非阻塞	MCP 工具执行期间，同 Worker 的其他 Episode 可继续推进
Reward 计算	异步并行	WeightedSum 中多个信号源（如 RuleReward + LLMJudge）可并行计算

资源隔离策略：

资源	隔离方式	说明
环境实例	每实例独立对象	Episode 间不共享环境实例，从预热池获取
内存	实例级隔离	环境实例的内部状态互不干扰
网络	后端决定	Process 后端共享网络，Podman 后端网络隔离
GPU	时间片共享	多个 Episode 的 GPU 推理请求排队执行

Worker 与 UEnv Server 的交互：

交互	方向	协议	说明
注册	Worker → Server	gRPC RegisterWorker	启动时上报 worker_id、supported_envs、capacity
心跳	Worker ↔ Server	gRPC WorkerHeartbeat (双向流)	定期保活，Server 下发控制指令
接收任务	Server → Worker	gRPC DispatchEpisode	接收 EpisodeRequest，返回 StreamReport
上报结果	Worker → Server	gRPC ReportResult	返回 EpisodeResult
健康检查	Worker → Worker	gRPC HealthCheck	自我健康检查

7.2 后端引擎

Backend trait 定义了环境实例生命周期管理的统一接口，向 Worker 屏蔽底层运行时差异。

核心方法：

方法	功能	说明
<code>create(spec)</code>	创建环境实例	返回实例信息（ID、端点、状态）
<code>destroy(instance_id)</code>	销毁环境实例	释放资源
<code>health_check(instance_id)</code>	健康检查	探测实例是否存活
<code>list_instances()</code>	列举所有受管实例	用于监控和管理
<code>capabilities()</code>	声明后端能力	返回 BackendCapabilities

BackendCapabilities 能力描述：

--	--

字段	说明
containerized	是否提供容器化隔离
rootless	是否支持 rootless 运行
gpu_support	是否支持 GPU
distributed	是否支持分布式部署
multi_agent	是否支持多 Agent
startup_latency_ms	启动延迟（毫秒）
max_concurrent_instances	最大并发实例数

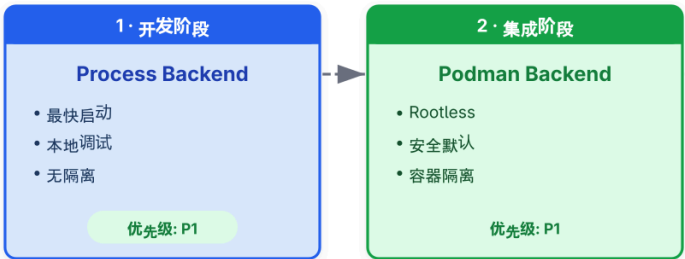
两后端对比：

后端	启动延迟	隔离级别	资源占用	适用场景	独特优势
Process	< 10ms	进程级	最低	开发调试、CI 测试	最快启动，即时反馈，零依赖
Podman	~2s	容器级(rootless)	低	生产默认、安全隔离	Rootless + Seccomp + AppArmor

各后端特点：

- **ProcessBackend**（MVP）：通过启动本地子进程运行环境实例，使用 DashMap 管理进程生命周期。无网络隔离和资源限制，但启动延迟最低。限制：无网络隔离、无资源限制、最大并发 100 实例。
- **PodmanBackend**（生产）：通过 Podman API 管理容器的创建/启停/销毁，每个环境实例运行在独立的 rootless 容器中。安全特性：rootless 运行、seccomp 系统调用白名单、AppArmor 强制访问控制、CPU/内存/GPU 资源约束。最大并发 1000 实例。

7.3 渐进部署路径



代码块

1	开发阶段	————▶	生产阶段
2	Process		Podman
3	Backend		Backend
4	• 最快启动		• Rootless
5	• 本地调试		• 安全默认
6	• 无隔离		• 容器隔离

八、数据结构

本章汇总系统中的所有核心数据实体。聚焦 UEnv Server 内部数据结构、奖励配置，以及各实体之间的关系图。

8.1 UEnv Server 内部数据

本节定义 UEnv Server 运行时维护的核心数据结构，包括环境注册表条目（EnvRegistryEntry）、环境实例（EnvInstance）、Worker 注册信息（WorkerInfo）及其容量和指标、进行中的 Episode 跟踪记录（InFlightEpisode），以及调度决策的打分明细（SchedulingDecision/ScoreBreakdown）。这些数据结构构成了 UEnv Server 调度决策和状态管理的基础。

EnvRegistryEntry — 环境注册表条目：

字段	类型	说明
env_type	String	环境类型标识，如 "math"、"code"
env_class	TypeId	环境类引用
supported_backends	Vec<BackendType>	支持的后端列表
default_config	Json	默认环境配置
resource_requirements	ResourceReq	资源需求

version	String	环境版本
---------	--------	------

EnvInstance — 环境实例：

字段	类型	说明
instance_id	Uuid	实例唯一 ID
env_type	String	环境类型
worker_id	WorkerId	所属 Worker
backend	BackendType	所属后端类型
state	InstanceState	Warm/Active/Idle/Cooling/Evicting/Destroyed
created_at	Timestamp	创建时间
last_used_at	Timestamp	最后使用时间
episode_count	u32	已执行的 Episode 数
config	Json	实例配置
resources	ResourceUsage	当前资源占用

WorkerInfo — Worker 注册信息：

字段	类型	说明
worker_id	WorkerId	Worker 唯一 ID
endpoint	String	gRPC endpoint
supported_envs	Vec<String>	支持的环境类型列表
backend	BackendType	当前使用的后端
capacity	WorkerCapacity	容量信息
state	WorkerState	Ready/Busy/Draining/Terminated

last_heartbeat	Timestamp	最后心跳时间
metrics	WorkerMetrics	实时指标

WorkerCapacity — Worker 容量：

字段	类型	说明
max_concurrent_episodes	u32	最大并发 Episode 数
current_episodes	u32	当前执行中的 Episode 数
available_slots	u32	可用槽位 = max - current
cpu_available	f64	可用 CPU 核
memory_available_mb	u64	可用内存 MB
gpu_available	Vec<GpuInfo>	可用 GPU 列表

WorkerMetrics — Worker 指标：

字段	类型	说明
total_episodes_completed	u64	已完成 Episode 总数
total_episodes_failed	u64	已失败 Episode 总数
avg_episode_duration_ms	f64	平均 Episode 耗时
avg_model_latency_ms	f64	平均模型推理延迟
p99_model_latency_ms	f64	P99 模型推理延迟
last_error	Option<String>	最近一次错误信息

InFlightEpisode — 进行中的 Episode：

字段	类型	说明

request_id	String	请求 ID
env_type	String	环境类型
worker_id	WorkerId	执行 Worker
instance_id	Option<Uuid>	使用的环境实例
started_at	Timestamp	开始时间
deadline	Timestamp	截止时间 = started_at + timeout
priority	u8	优先级
retry_count	u8	已重试次数

SchedulingDecision — 调度决策：

字段	类型	说明
worker_id	WorkerId	选中的 Worker
score	f64	综合得分 0.0-1.0
score_breakdown	ScoreBreakdown	各维度得分明细

ScoreBreakdown — 打分明细：

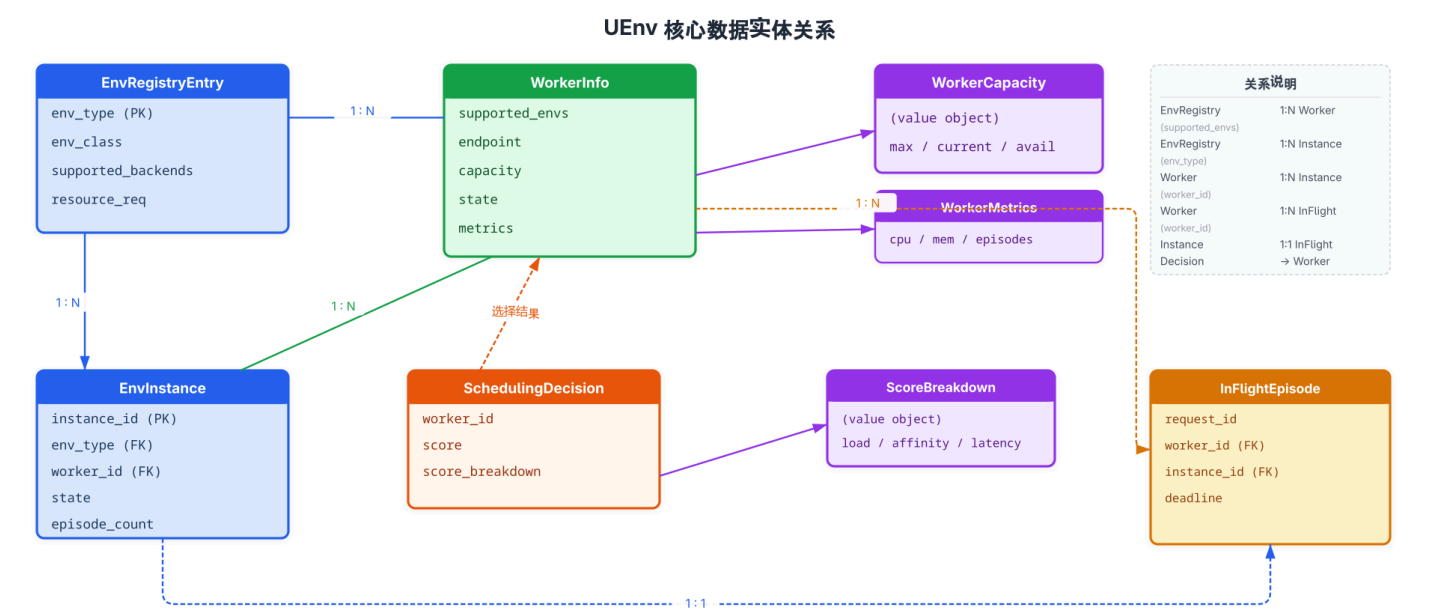
字段	类型	说明
load_score	f64	负载得分（权重 50%）
affinity_score	f64	亲和性得分（权重 30%）
latency_score	f64	延迟得分（权重 20%）

RewardConfig — 奖励计算配置：

RewardConfig 携带在 EpisodeRequest 中（见 § 5.2），定义了 Episode 执行时的奖励计算方式。
以下为完整字段说明。

字段	类型	说明
reward_type	string	"rubric"/"external"/"hybrid"
rubric_config	bytes	Sequential/Gate/WeightedSum 配置 (JSON)
external_endpoint	string	外部奖励端点 URL
reward_shaping	bytes	gamma/clip 等配置 (JSON)
component_weights	bytes	奖励组件权重 (JSON)

8.2 数据实体关系图



8.3 Protobuf 类型安全改进

结构化 message 定义：

```
代码块
1 // 替代 bytes env_config
2 message EnvConfig {
3     string env_type = 1;
4     map<string, string> params = 2; // 环境参数
```

```

5     repeated string dependencies = 3;           // 依赖的工具/资源
6     map<string, bytes> resources = 4;           // 嵌入式资源 (如 prompt template)
7     string env_id = 5;                         // 可选: 复用已有实例
8     int32 max_steps = 6;                      // 最大步数
9     int32 timeout_seconds = 7;                // 超时
10  }
11
12  // 替代 bytes reward_config
13  message RewardConfig {
14      string reward_type = 1;
15      repeated RewardLayer layers = 2;
16      RubricConfig rubric_config = 3;           // 替代 bytes rubric_config
17      map<string, float> weights = 4;          // 权重配置
18  }
19
20  message RubricConfig {
21      string rubric_type = 1;                   // "rule" | "model" | "llm_judge"
22      string rubric_model_endpoint = 2;        // 模型端点 (llm_judge 类型)
23      repeated Rule rules = 3;                 // 规则列表
24      map<string, string> rubric_params = 4;    // 通用参数
25  }
26
27  message RewardLayer {
28      string layer_type = 1;                    // "signal" | "credit" | "normalize"
29      string class_name = 2;                    // Python 类名
30      map<string, string> init_params = 3;      // 初始化参数
31  }
32
33  message Rule {
34      string name = 1;
35      string condition = 2;                     // 条件表达式
36      float reward_value = 3;
37  }
38
39  // TrajectoryStep 结构化
40  message TrajectoryStep {
41      int32 step_index = 1;
42      StructuredObservation observation = 2;
43      StructuredAction action = 3;
44      float reward = 4;
45      bool terminated = 5;
46      bool truncated = 6;
47      map<string, string> info = 7;
48  }
49
50  message StructuredObservation {
51      string content = 1;                       // 文本观测

```

```
52     map<string, float> metrics = 2;           // 数值指标
53     bytes raw_data = 3;                       // 非文本数据 (图像等)
54 }
55
56 message StructuredAction {
57     string action_type = 1;                   // "text" | "tool_call" | "code"
58     string content = 2;                       // 动作内容
59     repeated ToolCall tool_calls = 3;        // 工具调用 (Agent 场景)
60 }
61
62 message ToolCall {
63     string tool_name = 1;
64     string tool_call_id = 2;
65     map<string, string> arguments = 3;
66 }
```

8.4 错误码枚举

EpisodeError 的 `error_type` 从 `string` 改为枚举，并增加细粒度错误码和修复建议。

错误码枚举定义：

枚举值	编号	语义	典型原因	建议修复
ENV_ERROR	1	环境执行异常	环境代码 bug、资源不可用	检查环境日志，用 <code>uenv debug</code> 本地调试
MODEL_ERROR	2	模型推理异常	端点不可达、OOM、格式错误	检查模型端点状态和配置
TIMEOUT_ERROR	3	超时	Episode 执行超过 <code>timeout_seconds</code>	增大超时值或减少 <code>max_steps</code>
PROTOCOL_ERROR	4	协议错误	版本不兼容、消息格式错误	检查 <code>protocol_version</code> 和 <code>protobuf</code> 定义
RESOURCE_ERROR	5	资源不足	Worker 满载、内存不足	增加 Worker 数量或 <code>max_concurrent</code>

CONFIG_ERROR	6	配置错误	env_config 缺少必需字段	检查 EpisodeRequest 的 env_config 结构
INTERNAL_ERROR	7	内部错误	UEnv Server/Worker 内部 panic	查看 Server/Worker 日志，提交 issue

细粒度错误码（error_code 字段）：

代码块

```
1  格式：{ERROR_TYPE}.{SUB_CODE}
2
3  ENV_ERROR.INVALID_STATE      - 环境状态不正确（如 reset 前调用 step）
4  ENV_ERROR.TOOL_FAILED        - 工具调用失败
5  ENV_ERROR.PARSE_ERROR        - 环境输出解析失败
6  MODEL_ERROR.UNREACHABLE      - 模型端点无法连接
7  MODEL_ERROR.OOM              - 模型推理 OOM
8  MODEL_ERROR.INVALID_RESPONSE - 模型返回格式异常
9  TIMEOUT_ERROR.EPISODE        - Episode 整体超时
10 TIMEOUT_ERROR.MODEL_CALL      - 单次模型调用超时
11 RESOURCE_ERROR.WORKER_FULL    - Worker 已达 max_concurrent
12 RESOURCE_ERROR.NO_WORKER      - 无可用的 Worker（env_type 未注册）
```

EpisodeError 完整结构：

代码块

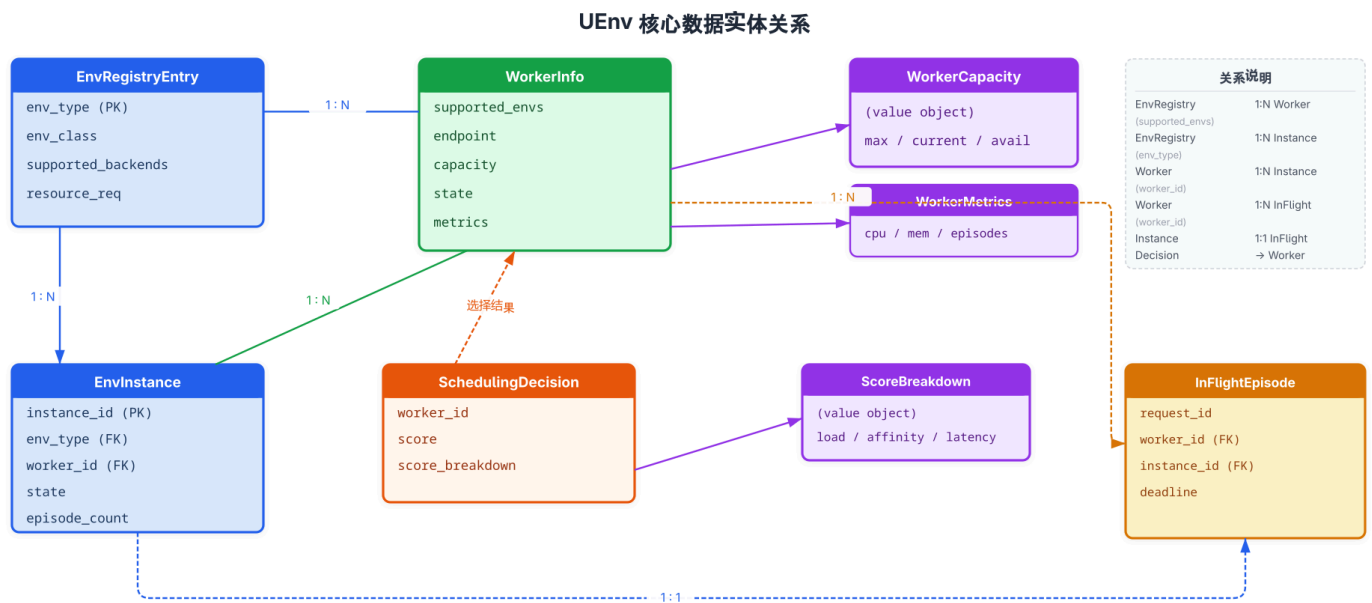
```
1  message EpisodeError {
2      ErrorType error_type = 1;           // 枚举：大类错误
3      string error_message = 2;           // 人类可读的错误描述
4      string error_code = 3;              // 细粒度错误码（如
      "ENV_ERROR.TOOL_FAILED")
5      string suggestion = 4;              // 用户可操作的修复建议
6      map<string, string> context = 5;     // 错误上下文（如 tool_name、
      model_endpoint）
7      string stack_trace = 6;             // 仅在 DEBUG 日志级别记录
8  }
```

九、状态机

本章汇总所有实体的生命周期状态转换。核心设计思想是"每个实体有明确的生命周期"——从创建到销毁，每个状态转换都有清晰的触发条件和执行动作，确保系统行为可预测。

9.1 Episode 状态机

Episode 从提交到完成经历 10 个状态和 15 条转换:



转换条件列表:

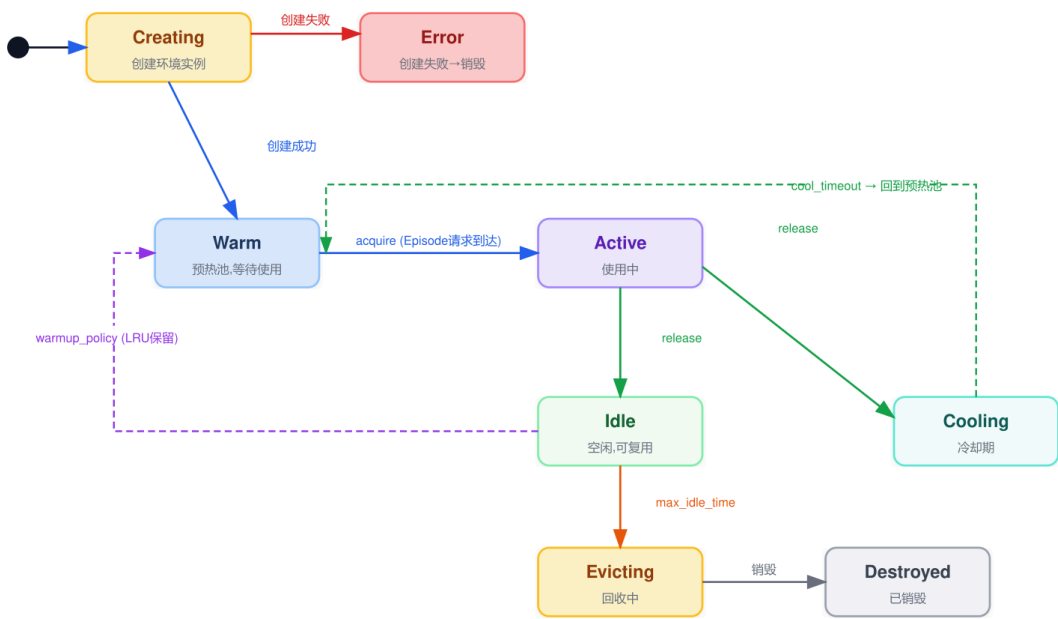
#	转换	触发条件
1	Pending → Scheduled	调度器选出 Worker
2	Scheduled → Idle	Worker 确认接收
3	Idle → Running	env.reset() 完成
4	Running → StepExecuting	模型返回 action
5	StepExecuting → Rewarding	env.step() 返回
6	Rewarding → Running	非终止 → 继续循环
7	Rewarding → Completing	terminated ∨ truncated ∨ max_steps
8	Running → Retrying	model_timeout (重试 ≤ max_retries)

9	Retrying → Running	重试成功
10	Retrying → Failed	重试耗尽
11	Completing → Completed	TrajectoryReward 计算完成
12	Completing → Failed	Reward 计算错误
13	任何状态 → Timeout	now() > deadline
14	任何状态 → Cancelled	外部取消请求
15	Pending → Cancelled	cancel / timeout

9.2 环境实例状态机

环境实例在实例池中的完整生命周期：

环境实例状态机



各状态含义：

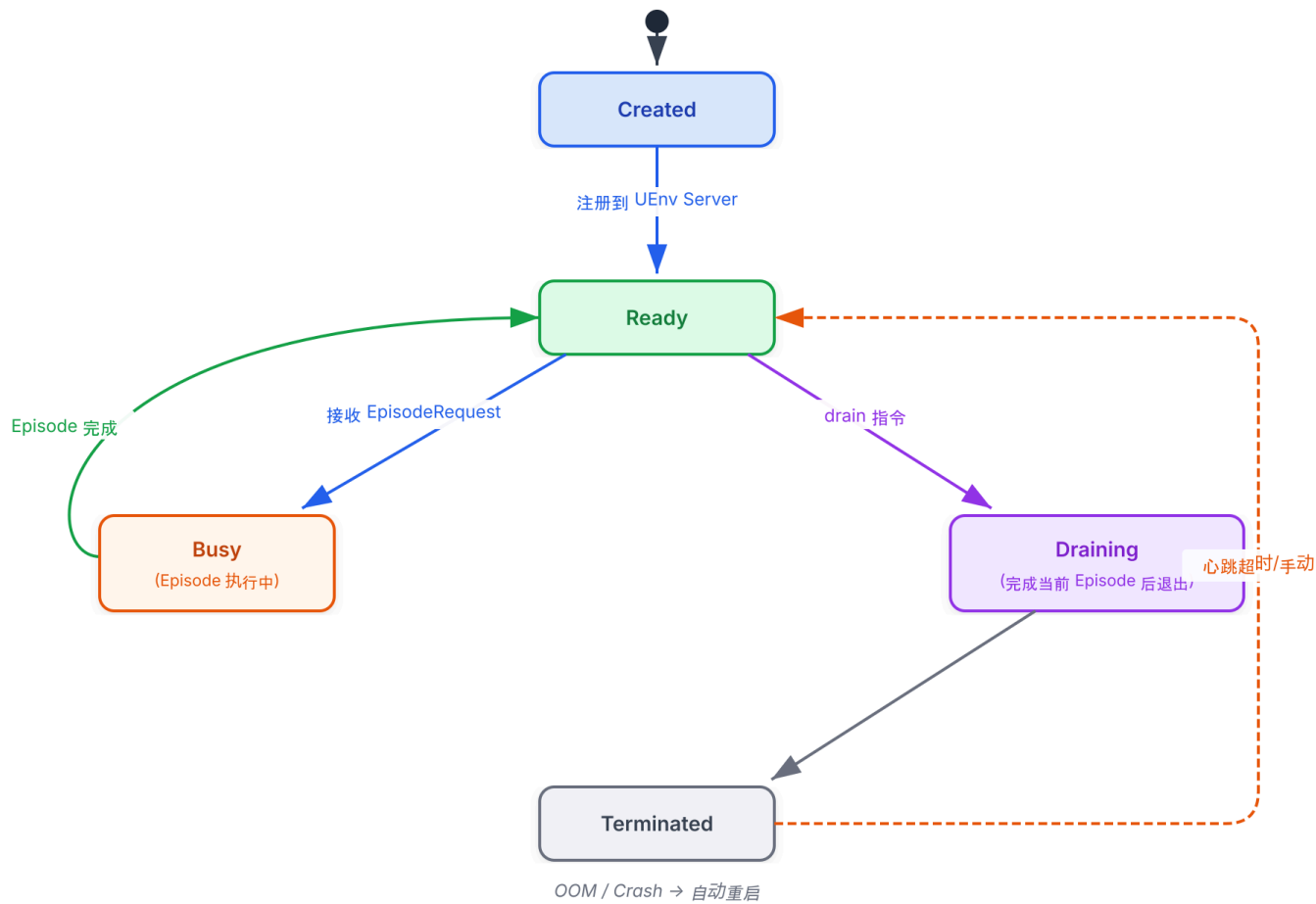
状态	含义
Creating	正在创建环境实例，初始化中

Warm	预热池中，等待被 Episode 使用
Active	正在被某个 Episode 使用
Idle	Episode 完成，空闲可复用
Cooling	冷却期，等待回到预热池
Evicting	被回收中，即将销毁
Destroyed	已销毁，资源已释放

实例池的配置参数（warmup_pool_size、max_idle_time、cool_timeout、max_episode_count）详见 § 6.4 环境预热池。

9.3 Worker 生命周期状态机

Worker 生命周期状态机

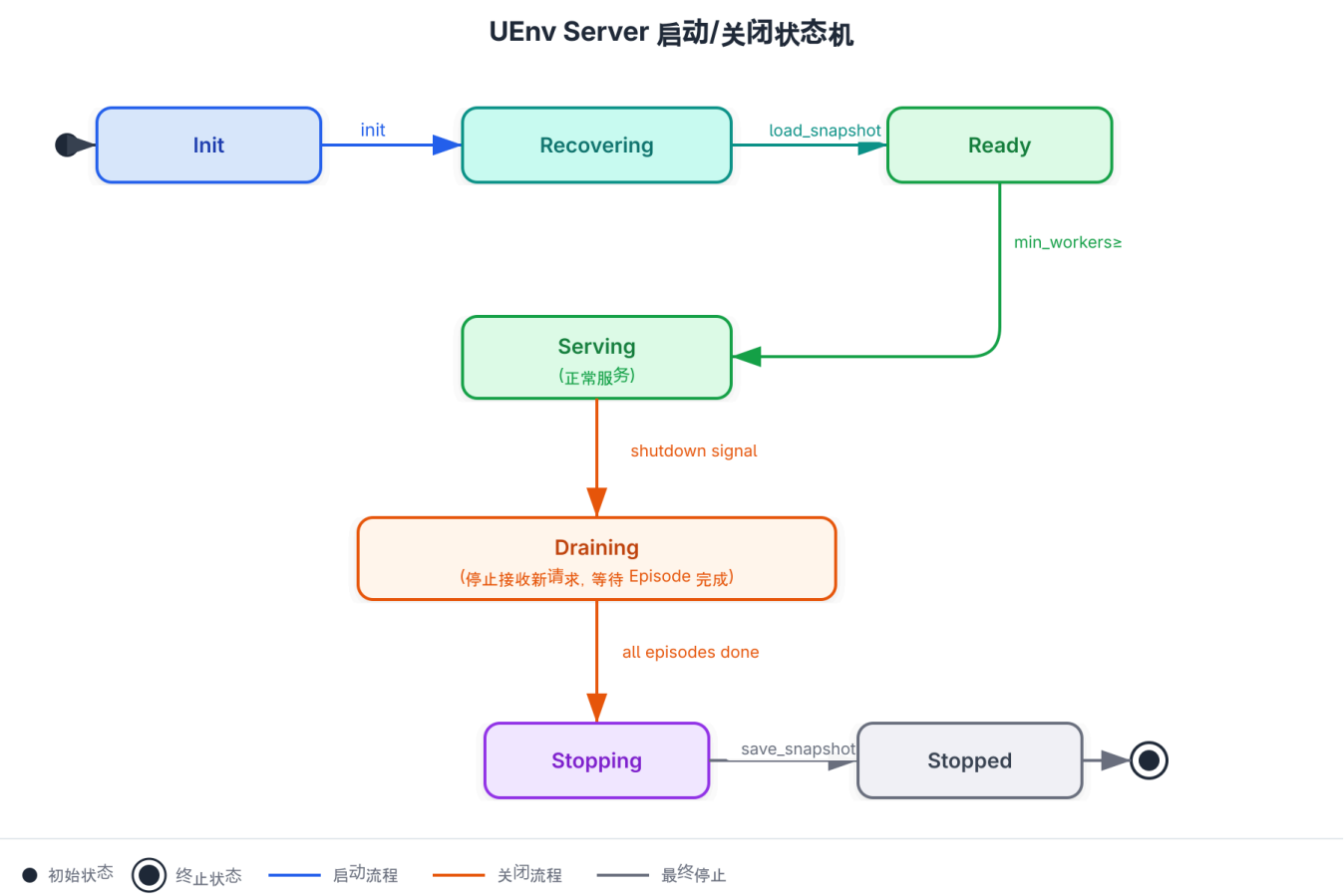


● 初始状态 — 主流程 — 循环回复 — 退出流程 - - - 异常恢复

状态说明：

状态	含义
Created	Worker 进程已启动，尚未注册
Ready	已注册到 UEnv Server，等待接收 EpisodeRequest
Busy	正在执行 Episode
Draining	收到 drain 指令，等待当前 Episode 完成后退出
Terminated	已终止（正常退出或故障）

9.4 UEnv Server 启动/关闭状态机



状态说明：

状态	含义

Init	加载配置、初始化 gRPC Server
Recovering	从快照恢复环境注册表和实例池状态
Ready	等待 Worker 注册（最小可用 Worker 数达标后转 Serving）
Serving	正常接收和调度 Episode 请求
Draining	优雅关闭，等待进行中 Episode 完成
Stopping	保存快照、关闭连接
Stopped	完全停止

状态转换触发条件：

转换	触发条件	动作
Init → Recovering	配置加载完成	读取最近快照文件
Recovering → Ready	快照恢复完成（或无快照直接就绪）	启动 Worker 心跳检测
Ready → Serving	注册 Worker 数 $\geq$ min_workers	开始接收 EpisodeRequest
Serving → Draining	收到 SIGTERM / shutdown 命令	停止接收新请求
Draining → Stopping	in_flight_episodes == 0	保存状态快照
Stopping → Stopped	快照保存完成	关闭 gRPC Server

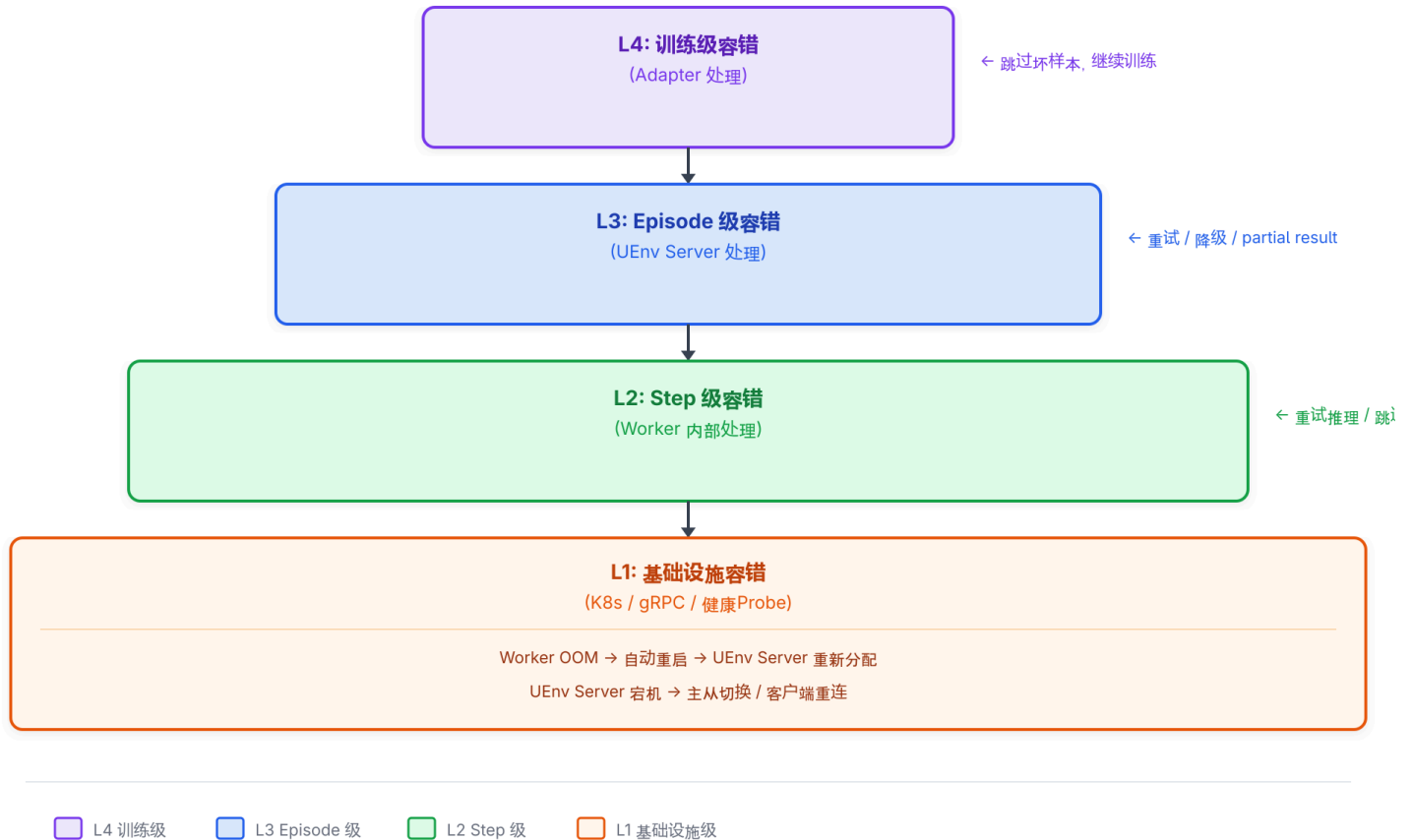
十、容错与可靠性

本章描述各层如何处理故障。核心设计思想是"错误应尽可能在离发生点最近的位置被处理"——形成四层金字塔，底层处理基础设施级故障，上层逐级处理更精细的错误，确保大部分错误在底层就被消化。

10.1 四层容错金字塔

四层容错金字塔

错误应尽可能在离发生点最近的位置被处理



各层设计思想：

- L1 基础设施层：**依赖 gRPC 的连接管理（自动重连）。Worker OOM 由进程管理器自动重启，UEnv Server 通过心跳检测到后重新分配 Episode。
- L2 Step 级：**Worker 内部处理单步执行失败。模型推理超时重试（最多 max_retries 次，每次 +2s 退避），工具调用失败跳过并记录错误信息。
- L3 Episode 级：**UEnv Server 处理整个 Episode 的失败。重新分配到其他 Worker（最多 3 次），或返回 partial_trajectory 让训练侧决定。
- L4 训练级：**Training Adapter 将失败 Episode 标记为坏样本，跳过继续训练，不中断训练流程。

10.2 重试策略

本节定义 UEnv 在各类故障场景下的重试策略，包括重试次数限制、超时设置和退避算法。核心原则是：可恢复的瞬态故障（网络抖动、模型推理超时）自动重试，不可恢复的故障（配置错误、环境异常）立即返回错误，避免无效等待。

场景	重试策略	超时	退避

Worker 端 Episode 失败	UEnv Server 重新分配到 其他 Worker	3 次	立即重试
模型推理失败	Worker 内部重试	max_retries 次	每次 +2s 退避
UEnv Server 不可用	gRPC 客户端重连	自动	指数退避
Worker OOM	进程管理器重启 + UEnv Server 摘除	自动	—
Episode 超时	返回 partial_trajectory	timeout_seconds	无重试
环境错误	返回 FAILED + partial_trajectory	—	无重试
配置错误	立即返回 FAILED	—	无重试

10.3 数据完整性

本节描述 UEnv 如何保证 EpisodeResult 中轨迹数据的完整性。通过 SHA-256 校验和机制，在 Worker 发送前计算轨迹数据的哈希值，训练侧收到后验证校验和是否匹配，确保数据在传输过程中不被篡改或损坏。

- 每个 EpisodeResult 包含 trajectory 的 **SHA-256 校验和**（trajectory_checksum 字段）
- Worker 端在发送前计算校验和，设置 integrity_verified = true
- Training Adapter 收到后验证完整性
- 校验失败视为 FAILED，按重试策略处理

10.4 UEnv Server 高可用

本节描述 UEnv Server 的状态持久化方案和可选的主从切换机制。UEnv Server 的状态分为可恢复状态（环境注册表、Worker 列表）和瞬态状态（进行中的 Episode），前者通过定期快照和 Worker 重注册恢复，后者通过超时机制清理。主从切换为可选功能，适用于对可用性要求极高的生产环境。

状态分类与恢复策略：

状态类型	存储方式	恢复策略	恢复时间
环境注册表			< 10s

	内存 + 定期快照（每 30s）	Worker 重新注册 + UEnvHub 拉取	
Worker 列表 + 健康状态	内存	Worker 心跳自动重建	< 心跳周期（5s）
环境实例池状态	内存 + 定期快照	从快照恢复 + Worker 上报实际状态	< 30s
进行中的 Episode	内存 WAL	超时后标记 FAILED，由训练侧重试	立即
调度队列	内存	排队中的请求由训练侧重重新提交	立即

主从切换（可选）：

主从切换采用热备模式，从节点实时同步主节点状态但不处理请求，仅在主节点故障时提升为主。切换流程如下：

阶段	主节点	从节点
正常运行	处理所有请求，定期快照	同步快照 + 实时转发心跳
故障检测	无响应	健康探测连续 3 次失败
提升为主	—	加载最新快照，启动 gRPC Server
状态补全	—	接受 Worker 注册，重建路由表
恢复服务	—	正常接收 EpisodeRequest

- **RPO**：最近一次快照后的增量状态，切换后通过 Worker 重新注册补全
- **部署要求**：主从节点需共享存储（快照文件），或从节点通过日志复制同步

10.5 网络分区处理

当 Worker ↔ Server 之间发生网络分区时，需要保证 Episode 数据不丢失、系统状态一致。

分区检测：

代码块

- ```
1 正常： Worker ←——心跳——→ Server（5s 间隔）
2 检测： 心跳超时 > 3 × interval → 标记 Worker 为 SUSPECT
```

Worker 侧行为：

- 1. 检测到 gRPC 连接断开 → 立即暂停接受新 Episode
- 2. 正在执行的 Episode 继续执行至完成或超时
- 3. 完成的 EpisodeResult 写入**本地 WAL**（Write-Ahead Log）
- 4. 启动**指数退避重连**：1s → 2s → 4s → 8s → 16s → 30s（上限）
- 5. 重连成功后：
  - 发送本地 WAL 中缓存的 EpisodeResult
  - 同步 Worker 当前状态（活跃 Episode、实例池）
  - 恢复正常心跳

Server 侧行为：

- 1. 心跳超时 → 标记 Worker 为 **SUSPECT**
- 2. SUSPECT 状态下，Worker 的进行中 Episode 不重新分配（等待恢复）
- 3. 连续 3 次超时 → 标记 **PARTITIONED**，将 in-flight Episode 重新入队
- 4. Worker 重连 → 验证状态 → 冲突解决：
  - Episode 已重新分配：丢弃 Worker 的缓存结果（通知 Worker）
  - Episode 未重新分配：接受 Worker 的缓存结果

本地 WAL 格式：

代码块

|    |                       |                         |  |
|----|-----------------------|-------------------------|--|
| 1  |                       |                         |  |
| 2  | WAL Entry             |                         |  |
| 3  |                       |                         |  |
| 4  | timestamp (8 bytes)   | 2026-04-19 12:00:00.123 |  |
| 5  | episode_id (32 bytes) | "ep-abc123..."          |  |
| 6  | entry_type (1 byte)   | RESULT=1, STATUS=2      |  |
| 7  | data_length (4 bytes) | 1024                    |  |
| 8  | data (variable)       | protobuf EpisodeResult  |  |
| 9  | crc32 (4 bytes)       | checksum                |  |
| 10 |                       |                         |  |

## 10.6 脑裂防护

主从切换时需要防止旧主继续接受写操作（脑裂场景）。

### Fencing Token 机制：

代码块

```
1 正常：
2 Server-Primary (epoch=5) ←— Worker 心跳 —→ Worker
3
4 故障切换：
5 Server-Primary (epoch=5) ←— ✕ 网络断开
6 Server-Standby 提升为 Primary (epoch=6)
7 Server-Standby (epoch=6) ←— Worker 心跳 —→ Worker
8
9 脑裂防护：
10 Server-Old-Primary (epoch=5) 尝试接受 Episode
11 → Worker 检查 epoch=5 < 6 → 拒绝请求
12 → Server-Old-Primary 检测到 epoch 过期 → 自动降级为 Standby
```

### 实现细节：

- **Lease 模型**：Primary 持有带 TTL 的 lease（如 15s），lease 过期自动降级
- **Worker 端验证**：每次心跳响应附带 `server_epoch`，Worker 发现 epoch 变化时重新注册
- **gRPC metadata**：每次请求携带 `x-uenv-server-epoch`，Server 校验 epoch 一致性
- **共享存储 fencing**：主从切换时在共享存储写入 fencing token，旧主检测到 token 变化即停止

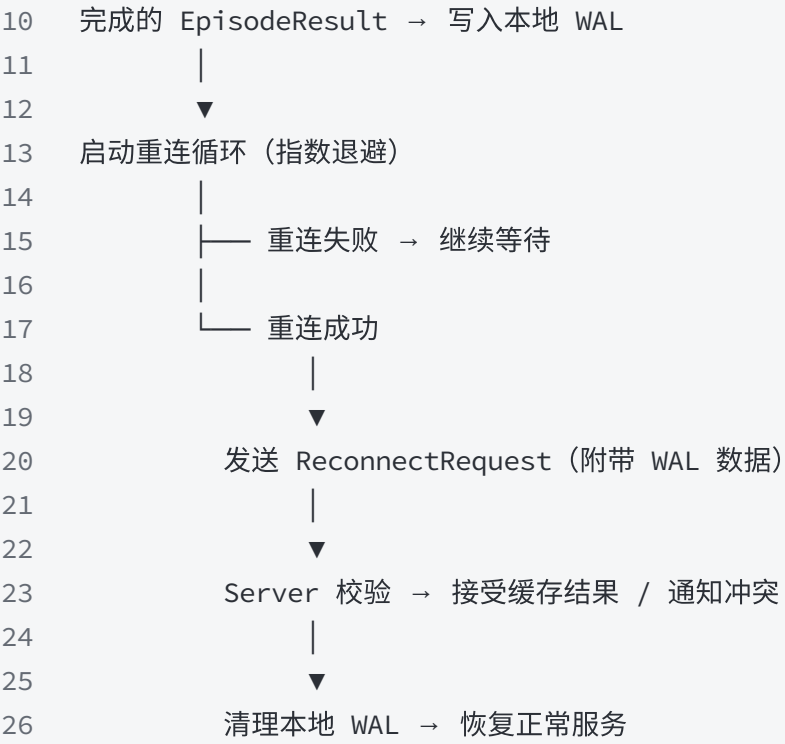
## 10.7 Worker 断连策略

Worker 断开与 Server 的连接后，需要保证已执行 Episode 的结果不丢失。

### 断连恢复流程：

代码块

```
1 Worker 断连检测
2 |
3 ▼
4 暂停接受新 Episode —→ 通知本地 EpisodeExecutor
5 |
6 ▼
7 正在执行的 Episode 继续至完成/超时
8 |
9 ▼
```



退避参数：

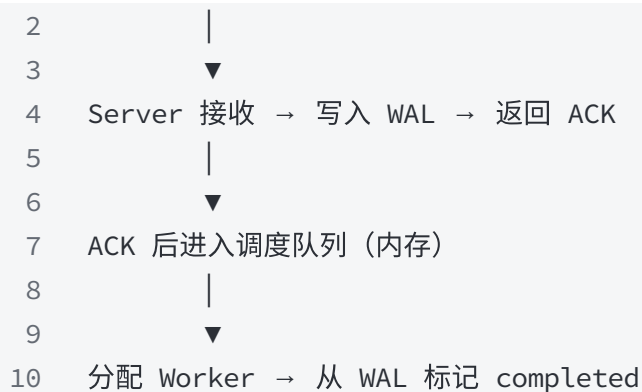
| 参数              | 值      | 说明                    |
|-----------------|--------|-----------------------|
| initial_backoff | 1 s    | 首次重连等待                |
| max_backoff     | 30 s   | 最大等待时间                |
| multiplier      | 2.0    | 每次翻倍                  |
| jitter          | ±20%   | 随机抖动防止惊群              |
| max_attempts    | 60     | 最多重试 60 次（约 30 分钟）    |
| max_wal_size    | 100 MB | 本地 WAL 最大大小，超限后丢弃最旧记录 |

10.8 请求持久化

持久化策略：

代码块

1 Adapter 提交 EpisodeRequest



## WAL 分层：

| 层级           | 存储                    | 适用场景            |
|--------------|-----------------------|-----------------|
| L1: 内存队列     | 内存                    | 正常运行（最低延迟）      |
| L2: 本地 WAL   | 磁盘文件                  | Server 崩溃恢复     |
| L3: 外部队列（可选） | Redis Streams / Kafka | 多 Server 共享、跨机房 |

## 崩溃恢复：

1. Server 启动 → 检查 WAL 文件
2. 重放所有 `status=PENDING` 的 EpisodeRequest
3. 重新入队等待调度
4. 忽略已 `COMPLETED` 的记录
5. 恢复完成后开始接受新请求

## Adapter 端确认机制：

- Adapter 调用 `SubmitEpisode` → Server 写入 WAL → 返回 `ACK { episode_id, accepted: true }`
- Adapter 收到 ACK 后才认为提交成功
- 未收到 ACK 的请求由 Adapter 侧重试

## 10.9 GC 暂停与心跳隔离

Rust/VM 环境的 GC 暂停可能导致心跳延迟，触发误判。



隔离策略：

- Worker 心跳在**独立线程**发送，与 Episode 执行线程隔离
- 心跳线程仅做 `send_heartbeat()` + `sleep(interval)`，不持有 GIL
- 心跳内容极简 (`worker_id + active_count + timestamp`)，无序列化开销

Server 端容忍策略：

- 心跳超时阈值 = `3 × 心跳间隔`（容忍 2 次连续丢失）
- SUSPECT 状态等待额外 1 个超时周期才标记 PARTITIONED
- 示例：5s 间隔 → 15s 超时 → SUSPECT → 20s → PARTITIONED

## 十一、性能分析

本章从延迟和吞吐两个维度分析 UEnv 系统的性能特征，量化框架引入的开销并验证其在生产场景下的可行性。首先通过全链路延迟分解，将单 Episode 的总延迟拆解为协议转换、调度分发、环境执行和序列化四个阶段，定位到框架开销仅占总延迟的 ~0.2%（约 2ms），远低于环境执行本身的时间消耗。其次，通过数学、代码、Agent 三个典型训练场景的吞吐量估算，验证系统在 4-8 Worker、并发 Episode 数 2-8 的配置下可达到 40-320 episodes/min 的吞吐能力。最后，将 v6.2 的 gRPC 方案与 v4.0 的 Redis Streams 方案进行多维度性能对比，量化 protobuf 序列化（6x 提升）、消息延迟（2.5x 提升）等方面的收益。本章所有性能数据基于架构设计推导，具体数值需在实现阶段通过基准测试验证。

### 11.1 延迟预算

单 Episode 从提交到返回的全链路延迟分解：

代码块

```
1 总延迟 = T_adapter + T_dispatch + T_episode + T_serialize
2
3 T_adapter ~0.5ms 协议转换（框架原生 → protobuf）
4 T_dispatch ~1.0ms UEnv Server 调度 + gRPC 转发
5 T_episode ~500ms-30s 环境执行（N步 × 模型推理）
6 |—— env.reset() ~5ms
7 |—— model.generate() ~100ms-5s（主要瓶颈）
8 |—— env.step() ~1-100ms（视工具调用）
9 |—— reward() ~1ms-2s（视信号源）
10 T_serialize ~0.5ms protobuf 序列化/反序列化
```

11

12     框架开销占比: (T\_adapter + T\_dispatch + T\_serialize) / T\_episode

13                 = ~2ms / ~1s ≈ 0.2%

## 11.2 吞吐量估算

本节基于不同训练场景的参数配置（Episode 数、Worker 数、并发度），估算 UEnv 系统在典型工作负载下的吞吐能力。估算结果用于指导资源规划——在给定训练规模下需要部署多少 Worker 和 GPU 节点。

| 场景       | Episode 数 | Worker 数 | 并发 Episode/Worker | 预估吞吐              |
|----------|-----------|----------|-------------------|-------------------|
| 数学训练     | 10,000    | 4        | 8                 | ~320 episodes/min |
| 代码训练     | 5,000     | 8 (GPU)  | 4                 | ~120 episodes/min |
| Agent 训练 | 1,000     | 4        | 2                 | ~40 episodes/min  |

# 十二、使用场景

本章通过 5 个端到端使用场景，展示 UEnv 在实际训练工作流中的集成方式和使用体验。场景覆盖三大主流训练框架的典型训练任务：ROLL + MathEnv 的数学推理训练、VeRL + CodeEnv 的代码生成训练、NeMo-RL + AgentEnv 的 Agent 工具使用训练。此外还包含两个横向场景——自定义环境的开发与发布流程，以及使用 OpenRLHF 进行大规模并行训练的运维场景。每个场景包含场景描述、端到端交互流程和简短的调用示意，帮助读者快速理解 UEnv 的使用方式和各组件的协作关系。这些场景同时也是系统验收的重要参考，验证了从训练框架发起请求到环境执行并返回结果的全链路连通性。建议读者结合 § 二 系统设计总览中的架构全貌图和 § 五 交互机制中的时序图对照阅读。

## 12.1 数学训练环境（ROLL + MathEnv）

**场景描述：** 使用 ROLL 框架训练数学推理模型，通过 GEMAdapter 将 ROLL 的 GEM 调用自动转换为 UEnv Episode。

**端到端流程：**

1. 训练工程师配置 GEMAdapter 的 UEnv endpoint
2. ROLL 发起 GEM task dict (env\_name="math", difficulty="hard")
3. GEMAdapter 转换为 EpisodeRequest → UEnv Server 调度到 MathEnv Worker
4. Worker 执行 Episode：模型生成答案 → MathEnv 验证 → Reward 计算
5. EpisodeResult 经 GEMAdapter 转换为 GEM result dict 返回 ROLL

**调用方式：** ① 创建 GEMAdapter 实例并指定 UEnv Server endpoint（如 dns:///uenv-server:50051）；② 调用 execute\_episode 方法，传入 env\_name ("math")、config (difficulty="hard")、model\_endpoint (vLLM 地址) 和 max\_steps (10)；③ 异步等待返回包含 trajectory 和 reward 的结果字典。

## 12.2 代码训练环境 (VeRL + CodeEnv)

**场景描述：** 使用 VeRL 框架训练代码生成模型，批量提交 DataProto 样本并流式回收结果。

**端到端流程：**

1. VeRL 训练循环产生 DataProto 批量样本
2. VeRLAdapter 将每个 sample 独立转换为 EpisodeRequest
3. 8 个 GPU Worker 并行执行代码沙箱环境（编译 + 单元测试）
4. 流式回收结果，Reward = Sequential(编译门控 + 测试通过率 + LLMJudge)

**调用方式：** ① 创建 VeRLAdapter 实例并指定 UEnv Server endpoint；② 将 DataProto 批量样本传入 execute\_batch 方法；③ 通过异步迭代器流式接收每个样本的 trajectory + reward 结果。

## 12.3 Agent 训练环境 (NeMo-RL + AgentEnv)

**场景描述：** 使用 NeMo-RL 做 Agent 工具使用训练，AgentEnv 支持多轮 Episode 和 MCP 工具调用。

**端到端流程：**

1. NeMoAdapter 将 BatchedDataDict + Ray Actor handle 转换为 EpisodeRequest
2. AgentEnv 执行多轮 Episode：模型生成 action → 工具调用（MCP 自动路由）→ 环境响应
3. 支持多工具：bash、write\_file、search 等
4. StreamReport 实时上报每步进度

## 12.4 自定义环境开发与发布

**场景描述：** 环境开发者创建安全分析环境，注册到 UEnvHub，所有训练框架可用。

**端到端流程：**

1. 实现 MCPEnvironment 子类，注册 scan\_code、query\_cve 工具
2. 实现 `_step_impl()` 方法定义环境逻辑
3. `@register_env("security")` 注册环境类型
4. 打包发布到 UEnvHub，Worker 拉取后即可调度

**开发步骤：** ① 定义 MCPEnvironment 子类；② 通过 `@tool_register` 装饰器注册 scan\_code、query\_cve 等工具（每个工具声明名称、描述和参数 Schema）；③ 实现 `_step_impl` 方法定义环境逻辑（普通 action 在此处理，call\_tool 自动路由到注册的工具函数）；④ 通过 `@register_env("security")` 注册环境类型名。

## 12.5 大规模并行训练

**场景描述：** 使用 OpenRLHF 进行 Agent 训练，并发提交 1000 个 Episode，由 UEnv Server 自动调度和重试。

**端到端流程：**

1. OpenRLHFAdapter 将训练批量转换为并行 EpisodeRequest
2. UEnv Server 调度到 Worker Pool（自动负载均衡 + 类型亲和）
3. Worker OOM 时自动重启，Episode 重新分配
4. 成功率 > 99%（含 3 次重试 + Worker 故障转移）

**调用方式：** ① 创建 OpenRLHFAdapter 实例并指定 UEnv Server endpoint；② 将训练批量中的每个请求构造为 execute\_episode 调用；③ 通过异步并发（如 asyncio.gather）同时提交 1000 个 Episode；④ 收集结果，成功约 992 个、失败约 8 个（自动重试 3 次后仍失败），失败样本标记为坏样本跳过。

## 十三、演进路线

本章规划 UEnv 从最小可行产品（MVP）到生产级系统的四阶段实施路线，基于架构可行性评审结论（详见附录 B）调整。核心原则是：**Phase 0 只做一件事 —— 用最简实现验证 ROLL + Sokoban 端到端跑通**。不验证的组件（WAL、心跳、多后端、Auth）全部砍掉，放到 Phase 1+。 "如果你没有加回至少 10% 被删的东西，说明删得不够。"

## 13.1 实现阶段

| Phase   | 时间         | 核心交付       | Adapter                           | 后端           |
|---------|------------|------------|-----------------------------------|--------------|
| Phase 0 | Week 1-4   | 架构验证       | GEMAdapter (ROLL)                 | Process（单进程） |
| Phase 1 | Week 5-10  | 双框架 + 可观测性 | +VeRLAdapter,<br>+TRLAdapter      | +Podman      |
| Phase 2 | Week 11-18 | 五框架 + 异步模式 | +NeMoAdapter,<br>+OpenRLHFAdapter | —            |
| Phase 3 | Week 19-26 | 高级环境 + 高性能 | 异步模式优化                            | —            |

### Phase 0 详细计划（4 周）：

| 周 | 范围                   | 交付物                                                            | 关键风险                                            |
|---|----------------------|----------------------------------------------------------------|-------------------------------------------------|
| 1 | Server 最小实现          | gRPC Service + 内存 HashMap 注册表 + 随机调度（无WAL/无心智/无Auth）           | —                                               |
| 2 | Worker 最小实现          | Episode 执行循环 + Python subprocess 后端 + 单 Worker 模式（无心跳/无预热池）    | env.step() 同步包装                                 |
| 3 | GEMAdapter + Sokoban | DataProto ↔ EpisodeRequest 转换 + Sokoban 环境 gRPC 封装             | <b>DataProto 序列化性能</b><br>（Week 3 立即 benchmark） |
| 4 | 端到端验证                | ROLL Agentic Pipeline → UEnv → Sokoban 完整训练循环 + 框架开销测量 + 清理/文档 | 端到端延迟验证                                         |

### Phase 0 验证标准（Go/No-Go 决策点）：

1.  ROLL 的 `TrajEnvManager` 通过 GEMAdapter 提交 Episode → UEnv 调度 → Worker 执行 → 结果返回 → ROLL 训练循环继续
2.  多步 Episode（Sokoban, 20 步）完整执行，轨迹数据完整

3. ✔ 端到端框架开销 < 10ms（不含环境执行和推理时间）
4. ✖ 以上任一未通过 → 暂停扩展，重新评估架构

对比 v7.0 原始标准：删除了"Worker 崩溃后 Episode 重新调度"——这在 Phase 0 单 Worker 模式下无意义。L3 Episode 级容错是 Phase 1+ 的范畴。

### 13.2 里程碑

代码块

1

Week 4

—►

M1：架构验证（ROLL + Sokoban 端到端，Go/No-Go 决策点）

2

Week 10

—►

M2：双框架支持（ROLL + VeRL + TRL）+ 可观测性 + WAL + 心跳

3

Week 18

—►

M3：五框架全覆盖 + 异步模式 + 状态外部化

4

Week 26

—►

M4：高级环境 + UEnvHub HTTP API + 1000 eps/s 吞吐

### 13.3 风险与缓解

风险清单基于架构可行性评审（附录 B）更新，

| 风险               | 概率 | 影响（Phase 0）               | 缓解措施                                                                                  |
|------------------|----|---------------------------|---------------------------------------------------------------------------------------|
| DataProto 序列化性能  | 高  | 高（直接影响跑通能力）               | Phase 0 Week 3 <b>立即</b> benchmark 验证；如不达标用 numpy bytes view 或 io.BytesIO 替代 protobuf |
| Adapter 复杂度被低估   | 中  | 高（Phase 0 唯一 Adapter）     | GEMAdapter 不做通用化设计，死磕 ROLL 单框架。优化一个不该存在的抽象，是最常见的工程错误                                  |
| 训练循环同步屏障缺失       | 低  | 低（Phase 0 单 Episode 不需要）  | ROLL 的 GroupQueueManager 处理批量收集。UEnv 不需要额外搞 SubmitBatch                               |
| UEnv Server 单点故障 | 低  | 低（单 Worker 不需要 Server HA） | 进程崩溃 → 训练侧重重试即可。不加 WAL 不加 HA                                                          |
| 推理 GPU 利用率低      | 高  | 低（Phase 0 只验证架构正确性，不优化性能） | Phase 2 再处理异步模式                                                                       |
|                  |    |                           |                                                                                       |

|            |   |                     |                                                          |
|------------|---|---------------------|----------------------------------------------------------|
| 模型推理端点管理简化 | 中 | 中                   | EpisodeRequest 的 model_endpoint 支持 Ray Actor（ROLL 的默认方式） |
| 框架协议变更     | 低 | 低（4 周内框架不太可能大改 API） | 保持 GEMAdapter 紧耦合                                        |
| 环境冷启动延迟    | 低 | 低（单 Worker 直接创建）    | Phase 1 再加预热池                                            |

## 十四、可观测性

本章定义 UEnv 的可观测性体系，覆盖日志、指标、追踪和告警四个维度。核心设计思想是"默认可观测"——所有组件在开发阶段就内置结构化日志和 Prometheus 指标，确保系统在部署后的运行行为完全透明。可观测性数据的设计遵循 OpenTelemetry 标准，便于与现有监控基础设施集成。

### 14.1 结构化日志

所有组件输出 JSON 格式结构化日志，支持日志聚合和检索。

日志格式：

代码块

```
1 {
2 "timestamp": "2026-04-19T12:00:00.123456+08:00",
3 "level": "INFO",
4 "logger": "uenv.server.scheduler",
5 "message": "Episode dispatched to worker",
6 "trace_id": "trace-abc123",
7 "span_id": "span-def456",
8 "context": {
9 "episode_id": "ep-xyz",
10 "worker_id": "worker-1",
11 "env_type": "math"
12 },
13 "hostname": "uenv-server-0",
14 "pid": 12345
15 }
```

日志级别策略：

| 级别    | 使用场景         | 示例                                   |
|-------|--------------|--------------------------------------|
| ERROR | 需要人工干预的异常    | Worker 崩溃、调度失败、WAL 写入失败              |
| WARN  | 可能影响性能但不中断服务 | 心跳超时、队列积压 > 50%、重试请求                 |
| INFO  | 正常业务流程关键节点   | Episode 提交/完成、Worker 注册/注销、Server 启动 |
| DEBUG | 调试详细信息       | 调度评分细节、Protobuf 序列化内容、心跳详情           |

组件日志覆盖率：

| 组件          | INFO 级覆盖                      | DEBUG 级覆盖                 |
|-------------|-------------------------------|---------------------------|
| UEnv Server | Episode 生命周期、Worker 生命周期、调度决策 | 评分算法细节、队列状态               |
| Worker      | Episode 开始/完成、实例创建/销毁         | env.step() 输入输出、Reward 计算 |
| Adapter     | 请求转换、提交确认                     | 原始协议内容、转换细节               |

14.2 Metrics 规范

UEnv 暴露 Prometheus 格式指标，Server 通过 `/metrics` 端点（默认 :9090）提供。

核心指标表：

| 指标名                                        | 类型        | 标签维度                                                                 | 说明                                         |
|--------------------------------------------|-----------|----------------------------------------------------------------------|--------------------------------------------|
| <code>uenv_episode_duration_seconds</code> | histogram | <code>env_type</code> , <code>worker_id</code> , <code>status</code> | Episode 执行耗时                               |
| <code>uenv_episode_total</code>            | counter   | <code>env_type</code> , <code>status</code>                          | Episode 计数 (status: completed/failed/timeo |



|                                          |           |                  |                                                |
|------------------------------------------|-----------|------------------|------------------------------------------------|
|                                          |           |                  | ut)                                            |
| uenv_worker_active_episodes              | gauge     | worker_id        | Worker 当前活跃 Episode 数                          |
| uenv_scheduler_queue_size                | gauge     | env_type         | 调度队列排队数                                        |
| uenv_scheduler_dispatch_duration_seconds | histogram | env_type         | 调度决策耗时                                         |
| uenv_worker_heartbeat_last_seen          | gauge     | worker_id        | Worker 最后心跳 Unix 时间                            |
| uenv_worker_total                        | gauge     | status           | Worker 总数 (status: active/suspect/partitioned) |
| uenv_instance_pool_size                  | gauge     | env_type, status | 环境实例池大小                                        |
| uenv_grpc_request_duration_seconds       | histogram | method, status   | gRPC 请求延迟                                      |
| uenv_grpc_request_total                  | counter   | method, status   | gRPC 请求计数                                      |
| uenv_wal_entries_pending                 | gauge     | —                | WAL 中待重放的条目数                                   |

使用示例：

代码块

```
1 # Episode 平均耗时
2 rate(uenv_episode_duration_seconds_sum[5m]) /
 rate(uenv_episode_duration_seconds_count[5m])
3
4 # Episode 失败率
5 rate(uenv_episode_total{status="failed"}[5m]) / rate(uenv_episode_total[5m])
6
7 # 调度队列积压
8 uenv_scheduler_queue_size
```

## 14.3 分布式追踪

UEnv 使用 `trace_id` 贯穿 Episode 全生命周期，兼容 OpenTelemetry。

### Trace 链路：

代码块

```
1 [Adapter] SubmitEpisode (trace_id=abc)
2 |
3 ▼ span: adapter.convert_request
4 [Server] Receive EpisodeRequest
5 |
6 ▼ span: server.schedule
7 [Server] Dispatch to Worker
8 |
9 ▼ span: worker.execute_episode
10 [Worker] Execute Episode (step loop)
11 | └─ span: worker.env_reset
12 | └─ span: worker.model_call (per step)
13 | └─ span: worker.env_step (per step)
14 | └─ span: worker.reward_compute
15 |
16 ▼ span: worker.report_result
17 [Server] Forward to Adapter
18 |
19 ▼ span: adapter.convert_response
20 [Adapter] Return to Training Framework
```

### Span 传播：

- `trace_id`：EpisodeRequest 中携带，Server → Worker → Result 全链路传递
- `span_id`：每个组件生成，上报到 OTLP collector
- gRPC metadata: `x-uenv-trace-id` + `x-uenv-span-id`

### 集成 OpenTelemetry（可选）：

代码块

```
1 from opentelemetry import trace
2
3 tracer = trace.get_tracer("uenv.server")
```

```

4
5 @tracer.start_as_current_span("schedule_episode")
6 def schedule_episode(self, request: EpisodeRequest):
7 span = trace.get_current_span()
8 span.set_attribute("env_type", request.env_type)
9 span.set_attribute("episode_id", request.episode_id)
10 # ... 调度逻辑

```

## 14.4 告警规则

基于 Prometheus AlertManager 的告警定义。

告警规则：

代码块

```

1 groups:
2 - name: uenv_critical
3 rules:
4 - alert: WorkerHeartbeatLost
5 expr: (time() - uenv_worker_heartbeat_last_seen) > 15
6 for: 1m
7 labels:
8 severity: P1
9 annotations:
10 summary: "Worker {{ $labels.worker_id }} 心跳丢失超过 15s"
11 action: "检查 Worker 进程状态, 可能需要重启"
12
13 - alert: ServerDown
14 expr: up{job="uenv-server"} == 0
15 for: 30s
16 labels:
17 severity: P1
18 annotations:
19 summary: "UEnv Server {{ $labels.instance }} 不可达"
20 action: "检查 Server 进程, 触发主从切换"
21
22 - name: uenv_warning
23 rules:
24 - alert: EpisodeFailureRateHigh
25 expr: >
26 rate(uenv_episode_total{status="failed"}[5m])
27 / rate(uenv_episode_total[5m]) > 0.05
28 for: 5m

```

```

29 labels:
30 severity: P2
31 annotations:
32 summary: "Episode 失败率超过 5%"
33 action: "检查环境代码和 Worker 日志"
34
35 - alert: SchedulerQueueBacklog
36 expr: uenv_scheduler_queue_size > 100
37 for: 5m
38 labels:
39 severity: P2
40 annotations:
41 summary: "调度队列积压 {{ $value }} 个请求"
42 action: "增加 Worker 或检查 Worker 健康"
43
44 - alert: ServerMemoryHigh
45 expr: process_resident_memory_bytes{job="uenv-server"} > 0.8 * 8 *
1024^3
46 for: 5m
47 labels:
48 severity: P2
49 annotations:
50 summary: "Server 内存使用超过 80%"
51 action: "检查实例池大小，考虑分片"

```

## 14.5 健康检查

| 端点                    | 用途              | 检查内容                 |
|-----------------------|-----------------|----------------------|
| <code>/healthz</code> | Liveness（存活检查）  | 进程存活、gRPC Server 可响应 |
| <code>/readyz</code>  | Readiness（就绪检查） | Worker 注册表非空、调度器就绪   |

健康检查响应：

代码块

```

1 {
2 "status": "healthy",
3 "version": "7.0.0",
4 "uptime_seconds": 3600,

```

```
5 "checks": {
6 "grpc_server": "ok",
7 "scheduler": "ok",
8 "worker_registry": {"count": 5, "status": "ok"},
9 "wal": {"pending": 0, "status": "ok"}
10 }
11 }
```

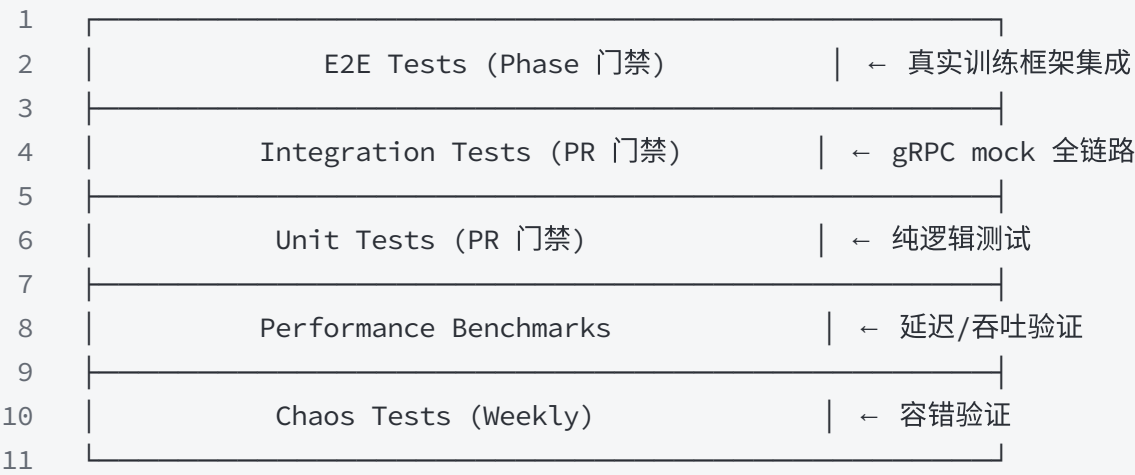
Worker 心跳即健康检查：Server 心跳响应附带 `server_status` (SERVING/DRAINING/STOPPING) 。

## 十五、测试策略与质量保证

本章定义 UEnv 的测试分层策略、覆盖率目标、性能基准和 CI/CD 流水线。

### 15.1 测试分层策略

代码块



### 15.2 单元测试规范

覆盖率目标：

| 模块 | 目标 | 说明 |
|----|----|----|
|    |    |    |

|                      |       |                                                  |
|----------------------|-------|--------------------------------------------------|
| Reward 系统            | ≥ 95% | 已有测试基础                                           |
| Adapter 协议转换         | ≥ 85% | 每个 Adapter 的<br>convert_request/convert_response |
| 调度评分算法               | ≥ 90% | 多维打分、过滤逻辑                                        |
| 状态机转换                | ≥ 90% | 每个实体的状态转换                                        |
| gRPC Service Handler | ≥ 80% | Mock Client 测试                                   |
| 整体                   | ≥ 80% | 全项目目标                                            |

## 工具栈：

- `pytest` + `pytest-asyncio` （异步测试）
- `grpcio-testing` （gRPC mock）
- `pytest-cov` （覆盖率）
- `hypothesis` （属性测试，用于调度算法）

## Mock 策略：

代码块

```

1 # Mock gRPC Channel
2 @pytest.fixture
3 def mock_uenv_stub():
4 channel = grpc.insecure_channel("localhost:0")
5 # 使用 grpc_testing 创建 mock server
6 from grpc_testing import server_from_dictionary
7 test_server = server_from_dictionary(
8 SERVICERS, grpc_testing.strict_real_time()
9)
10 yield test_server
11
12 # Mock 环境执行
13 @pytest.fixture
14 def mock_environment():
15 env = MagicMock(spec=BaseEnvironment)
16 env.reset.return_value = Observation(content="obs", metrics={})
17 env.step.return_value = StepResult(
18 observation=Observation(content="step-obs"),
19 reward=1.0, terminated=False, truncated=False
20)

```

### 15.3 集成测试方案

gRPC 全链路 Mock：

| 测试场景          | 输入                                                     | 验证                 |
|---------------|--------------------------------------------------------|--------------------|
| 完整 Episode 流程 | SubmitEpisode → Schedule → Dispatch → Execute → Report | EpisodeResult 正确返回 |
| Worker 注册+心跳  | RegisterWorker → HeartbeatStream                       | 注册表更新、心跳正常         |
| 多 Worker 调度   | 10 Worker + 100 Episode                                | 均衡分配、无丢失           |
| Adapter 转换    | 5 种框架格式                                                | 转换正确、round-trip 一致 |

5 个 Adapter 集成测试矩阵：

| Adapter         | 协议转换测试                                             | 端到端测试                   | 回归测试 |
|-----------------|----------------------------------------------------|-------------------------|------|
| GEMAdapter      | GEM → EpisodeRequest → GEM                         | mock GEM env_server     | ✓    |
| VeRLAdapter     | DataProto → EpisodeRequest → DataProto             | mock VeRL training loop | ✓    |
| NeMoAdapter     | BatchedDataDict → EpisodeRequest → BatchedDataDict | mock Ray Actor          | ✓    |
| OpenRLHFAdapter | env.step → EpisodeRequest → env.step result        | mock env.make/step      | ✓    |
| TRLAdapter      | MCP context → EpisodeRequest → MCP response        | mock MCP server         | ✓    |

## 15.4 端到端测试

每个 Phase 至少一个真实训练框架的完整训练循环。

| Phase   | 训练框架     | 后端             | 测试环境 | 验证目标                     |
|---------|----------|----------------|------|--------------------------|
| Phase 0 | ROLL     | ProcessBackend | 本地   | Episode 完整执行 + Reward 正确 |
| Phase 1 | VeRL     | PodmanBackend  | 单机容器 | 容器隔离 + 资源限制              |
| Phase 2 | NeMo-RL  | PodmanBackend  | 多机容器 | 分布式调度                    |
| Phase 2 | TRL      | PodmanBackend  | 多机容器 | MCP 工具调用                 |
| Phase 2 | OpenRLHF | PodmanBackend  | 多机容器 | 大规模并行                    |

### E2E 测试用例模板：

代码块

```
1 @pytest.mark.e2e
2 async def test_roll_math_training():
3 """ROLL + MathEnv 完整训练循环"""
4 # 1. 启动 UEnv Server (ProcessBackend)
5 # 2. 注册 MathEnv Worker
6 # 3. 通过 GEMAdapter 提交 10 个 Episode
7 # 4. 验证所有 Episode 返回正确的 Reward
8 # 5. 验证 Worker 状态正确更新
9 # 6. 关闭 Server
```

## 15.5 性能基准

| 基准名称                | 验证目标 | 通过标准                              |
|---------------------|------|-----------------------------------|
| latency_budget_test | 框架开销 | Adapter→Server→Worker 调度延迟 < 5ms  |
| throughput_test     | 最大吞吐 | 单 Server ≥ 500 eps/s (100 Worker) |



|                     |        |                                  |
|---------------------|--------|----------------------------------|
| serialization_bench | 序列化性能  | Protobuf 序列化比 JSON 快 $\geq 5x$   |
| grpc_stream_test    | 流式性能   | SubmitEpisodeStream 1000 eps 无背压 |
| heartbeat_overhead  | 心跳开销   | 100 Worker 心跳占用 $< 5\%$ CPU      |
| wal_throughput      | WAL 写入 | WAL 写入 $\geq 10000$ ops/s        |
| failover_test       | 故障恢复   | Server 主从切换 RTO $< 30s$          |

**回归阈值：** 任何基准测试结果退步  $> 10\%$  触发 CI 失败。

## 15.6 混沌测试计划

引入混沌测试，验证容错设计的有效性。

| 场景         | 注入方式                                          | 验证                        |
|------------|-----------------------------------------------|---------------------------|
| Worker OOM | <code>kill -9</code> Worker 进程                | Episode 重新调度、无数据丢失        |
| Server 崩溃  | kill Server 主节点                               | 主从切换 $< 30s$ 、Worker 自动重连 |
| 网络延迟       | <code>tc netem delay 500ms</code>             | Episode 延迟增加但不失败          |
| 网络分区       | iptables DROP Worker $\leftrightarrow$ Server | Worker 缓冲结果、重连后回传         |
| 磁盘满        | <code>dd if=/dev/zero</code> 填满磁盘             | WAL 写入失败告警、降级为纯内存         |
| 模型端点宕机     | 停止模型服务                                        | Episode 超时 + fallback     |

## 15.7 CI/CD 流水线

Code block

```
1 # .github/workflows/uenv-ci.yml
2 stages:
3 - lint # ruff + mypy + protobuf lint
4 - unit # pytest --cov ($\geq 80\%$)
5 - integration # gRPC mock 全链路测试
6 - build # Podman build + helm package
```

- 7

- e2e

# nightly: 真实训练框架测试
- 8

- perf

# weekly: 性能基准
- 9

- chaos

# weekly: 混沌测试

| 阶段          | 触发            | 时限       | 通过条件                    |
|-------------|---------------|----------|-------------------------|
| lint + unit | 每个 PR         | < 10 min | 0 error, coverage ≥ 80% |
| integration | 每个 PR         | < 20 min | 全部通过                    |
| build       | merge to main | < 5 min  | Podman image 构建         |
| e2e         | nightly (每天)  | < 2 hour | 至少 1 个框架 E2E 通过         |
| perf        | weekly        | < 1 hour | 无回归 > 10%               |
| chaos       | weekly        | < 2 hour | 全部容错场景通过                |

## 十六、开发者指南

本章面向三类用户：环境开发者（实现 Environment）、训练框架集成者（开发 Adapter）和运维工程师（部署管理）。

### 16.1 SDK 设计（uenv-sdk）

Crate 结构（Rust 2024 Edition）：

代码块

```
1 uenv-sdk/
2 |— Cargo.toml
3 |— src/
4 | |— lib.rs # crate 根: pub mod 导出
5 | |— environment/
6 | | |— mod.rs
7 | | |— traits.rs # Environment trait (见 TechSpec §2.1)
8 | | |— registry.rs # register_env!() 宏
9 | |— reward/
10 | | |— mod.rs
11 | | |— sequential.rs # Sequential, Gate, WeightedSum
12 | | |— sources.rs # RuleReward, NeuralRewardModel
```

```

13 | | └─ normalization.rs # ExponentialDiscounting 等
14 | └─ proto/
15 | | └─ mod.rs
16 | | └─ uenv.rs # prost!() 生成的 protobuf 代码
17 | └─ config.rs # EnvConfig, RewardConfig
18 └─ examples/
19 | └─ math_env.rs # 数学环境示例
20 | └─ code_env.rs # 代码执行环境示例
21 | └─ agent_env.rs # Agent 工具调用环境示例
22 └─ tests/
23 └─ integration.rs

```

## 核心 API (Rust 2024 trait + proc macro) :

代码块

```

1 ///! uenv-sdk/src/environment/traits.rs
2 ///!
3 ///! 环境开发者只需要实现 Environment trait 的 reset() 和 step(),
4 ///! 其余由 #[register_env] proc macro 自动生成。
5 ///!
6 ///! 使用 Rust 2024 Edition 语法:
7 ///! - impl Trait 参数
8 ///! - async fn in trait (RPITIT)
9 ///! - let ... else 模式
10 ///! - 嵌套 if-let 解构
11
12 use async_trait::async_trait;
13 use uenv_sdk_derive::register_env;
14
15 /// 环境类型标记: 由 #[register_env("name")] 自动实现
16 pub trait EnvType: Send + Sync + 'static {
17 fn env_type_name() -> &'static str;
18 }
19
20 /// 核心 Environment trait
21 ///
22 /// 生命周期: reset() → step() × N → close()
23 /// 线程安全: 同一实例一次只能被一个 Episode 使用
24 #[async_trait]
25 pub trait Environment: Send + 'static {
26 type Observation: Serialize + DeserializeOwned + Send;
27 type Action: Serialize + DeserializeOwned + Send;
28
29 /// 重置环境到初始状态

```

```

30 async fn reset(&mut self, seed: Option<u64>) -> Result<(Self::Observation,
EnvInfo)>;
31
32 /// 执行一步动作
33 async fn step(&mut self, action: Self::Action) ->
Result<StepResult<Self::Observation>>;
34
35 /// 清理环境资源 (可选)
36 fn close(&mut self) -> Result<()> { Ok(()) }
37 }
38
39 /// reset() 返回的环境元信息
40 #[derive(Serialize, Deserialize)]
41 pub struct EnvInfo {
42 pub suffix: Option<String>,
43 pub extras: HashMap<String, String>,
44 }
45
46 /// step() 返回的标准五元组
47 #[derive(Serialize, Deserialize)]
48 pub struct StepResult<Obs> {
49 pub observation: Obs,
50 pub reward: f64,
51 pub terminated: bool,
52 pub truncated: bool,
53 pub info: HashMap<String, String>,
54 }
55
56 /// #[register_env] proc macro 使用示例
57 ///
58 /// 此宏自动:
59 /// 1. 实现 EnvType trait → env_type_name() 返回 "math_verification"
60 /// 2. 生成 EnvironmentRegistry 的静态注册表条目
61 /// 3. 生成 derive 宏兼容的 Serialize/Deserialize
62 #[register_env("math_verification")]
63 pub struct MathEnv {
64 problem: String,
65 answer: String,
66 steps: usize,
67 max_steps: usize,
68 }
69
70 #[async_trait]
71 impl Environment for MathEnv {
72 type Observation = serde_json::Value;
73 type Action = serde_json::Value;
74

```

```

75 async fn reset(&mut self, seed: Option<u64>) -> Result<(Self::Observation,
EnvInfo)> {
76 self.steps = 0;
77 Ok((serde_json::json!({"content": self.problem}), EnvInfo::default()))
78 }
79
80 async fn step(&mut self, action: Self::Action) ->
Result<StepResult<Self::Observation>> {
81 self.steps += 1;
82 let terminated = self.steps >= self.max_steps;
83 // 处理 action、计算 reward 的逻辑...
84 let correct = action.get("answer").and_then(|v| v.as_str()) ==
Some(&self.answer);
85 let reward = if correct { 1.0 } else { 0.0 };
86 Ok(StepResult {
87 observation: serde_json::json!({"feedback": ""}),
88 reward,
89 terminated: terminated || correct,
90 truncated: self.steps >= self.max_steps,
91 info: HashMap::new(),
92 })
93 }
94 }
95
96 /// 内嵌 Python 环境桥接 (可选)
97 ///
98 /// 对于已有 Python 环境, 通过 pyo3 桥接: 将 Python 的 reset/step 暴露为
99 /// Environment trait 实现。
100 ///
101 /// ``ignore
102 /// use pyo3::prelude::*;
103 ///
104 /// pub struct PyEnvBridge {
105 /// py_env: Py<PyAny>,
106 /// }
107 ///
108 /// #[async_trait]
109 /// impl Environment for PyEnvBridge {
110 /// // ... 通过 Python 的 gil 调用 env.reset() / env.step()
111 /// }
112 /// ``

```

**Crate 版本策略 (Rust 2024 edition resolver = "3") :**

| Crate                        | 说明               | 关键依赖                                              |
|------------------------------|------------------|---------------------------------------------------|
| <code>uenv-sdk</code>        | 环境开发 SDK         | tonic, prost, serde, async-trait, uenv-sdk-derive |
| <code>uenv-server</code>     | Server 运行时       | uenv-sdk, tokio, tower, prometheus                |
| <code>uenv-worker</code>     | Worker 运行时       | uenv-sdk, tokio, pyo3 (可选 Python 桥接)              |
| <code>uenv-cli</code>        | 命令行工具            | uenv-sdk, clap, tokio                             |
| <code>uenv-sdk-derive</code> | proc-macro crate | syn, quote, proc-macro2                           |

## 16.2 CLI 工具 (uenv-cli)

uenv-cli 使用 clap 构建，编译为单一静态链接二进制文件。

代码块

```
1 # === 环境管理 ===
2 uenv env list # 列出已注册环境
3 uenv env test --local math_env.ron # 本地测试环境 (无需 Server)
4 uenv env publish --tag v1.0 # 发布到 UEnvHub (Phase 2+)
5
6 # === Episode 提交 ===
7 uenv submit --env-type math --input problem.json # 提交单个 Episode
8 uenv submit --batch inputs/ --output results/ # 批量提交
9
10 # === Worker 管理 ===
11 uenv worker list # 列出所有 Worker
12 uenv worker status <worker-id> # 查看 Worker 详情
13 uenv worker drain <worker-id> # 排空 Worker
14 uenv worker start --env-types math,code --max-concurrent 8 # 启动 Worker
15
16 # === 系统状态 ===
17 uenv status # 系统概览
18 uenv status --verbose # 详细状态 (含 Metrics)
19
20 # === 调试 ===
21 uenv debug --env-type math --input debug_case.json # 本地调试模式
22 uenv debug --replay episode-abc123.json # 重放 EpisodeRequest
```

Rust CLI 入口 (clap derive) :

## 代码块

```
1 ///! uenv-cli/src/main.rs
2 ///!
3 ///! 编译为静态链接二进制: cargo build --target x86_64-unknown-linux-musl -p uenv-
 cli
4
5 use clap::{Parser, Subcommand};
6
7 #[derive(Parser)]
8 #[command(name = "uenv", version, about = "UEnv 分布式环境框架 CLI")]
9 struct Cli {
10 #[command(subcommand)]
11 command: Command,
12 }
13
14 #[derive(Subcommand)]
15 enum Command {
16 /// 环境管理
17 Env {
18 #[command(subcommand)]
19 action: EnvAction,
20 },
21 /// Worker 管理
22 Worker {
23 #[command(subcommand)]
24 action: WorkerAction,
25 },
26 /// 提交 Episode
27 Submit {
28 #[arg(long)]
29 env_type: String,
30 #[arg(long)]
31 input: String,
32 #[arg(long)]
33 batch: Option<String>,
34 #[arg(long)]
35 output: Option<String>,
36 },
37 /// 系统状态
38 Status {
39 #[arg(long, default_value_t = false)]
40 verbose: bool,
41 },
42 /// 调试
43 Debug {
44 #[arg(long)]
```

```

45 env_type: String,
46 #[arg(long)]
47 input: Option<String>,
48 #[arg(long)]
49 replay: Option<String>,
50 #[arg(long, default_value_t = false)]
51 step_by_step: bool,
52 },
53 }
54
55 #[tokio::main]
56 async fn main() -> Result<(), Box<dyn std::error::Error>> {
57 let cli = Cli::parse();
58 match cli.command {
59 Command::Env { action } => handle_env(action).await?,
60 Command::Worker { action } => handle_worker(action).await?,
61 Command::Submit { env_type, input, batch, output } => {
62 handle_submit(env_type, input, batch, output).await?;
63 }
64 Command::Status { verbose } => handle_status(verbose).await?,
65 Command::Debug { env_type, input, replay, step_by_step } => {
66 handle_debug(env_type, input, replay, step_by_step).await?;
67 }
68 }
69 Ok(())
70 }

```

## 16.3 本地开发体验（Quick Start）

### 5 分钟 Hello World:

#### 代码块

```

1 # 1. 安装 CLI (Rust 静态链接二进制)
2 cargo install uenv-cli
3 # 或下载预编译二进制: https://uenv.dev/download
4
5 # 2. 创建环境 (使用模板)
6 uenv init my-env --template math
7 # 生成: src/math_env.rs, Cargo.toml, tests/integration.rs
8
9 # 3. 实现环境 (编辑 src/math_env.rs)
10 # ... 实现 Environment trait 的 reset() 和 step()
11
12 # 4. 本地测试

```



```

13 uenv env test --local src/math_env.rs
14 # ✓ reset() returned Observation
15 # ✓ step() returned StepResult
16 # ✓ Episode completed in 0.5s
17
18 # 5. 启动本地 Server + Worker
19 uenv server start --backend process &
20 uenv worker start --env-types my_env &
21 sleep 2
22
23 # 6. 提交 Episode
24 echo '{"problem": "1+1=?"}' | uenv submit --env-type my_env
25 # EpisodeResult: reward=1.0, status=COMPLETED
26
27 # 7. 停止
28 uenv server stop

```

## 环境开发模板项目结构：

### 代码块

```

1 my-env/
2 |— Cargo.toml # crate 定义
3 |— src/
4 | |— math_env.rs # Environment trait 实现
5 |— tests/
6 | |— integration.rs # 集成测试
7 |— config/
8 | |— reward.ron # Reward 配置（可选，RON 格式）
9 |— README.md # 环境说明

```

### 代码块

```

1 # my-env/Cargo.toml
2 [package]
3 name = "my-env"
4 version = "0.1.0"
5 edition = "2024"
6
7 [dependencies]
8 uenv-sdk = "0.1"

```

```

9 serde = { version = "1", features = ["derive"] }
10 serde_json = "1"
11 tokio = { version = "1", features = ["full"] }
12 anyhow = "1"
13
14 [lib]
15 crate-type = ["cdylib", "lib"] # 可编译为动态库供 uenv-worker 加载

```

## 16.4 调试工具

**uenv debug 模式（基于 tracing + tokio console）：**

代码块

```

1 # 本地单步调试
2 uenv debug --env-type math --step-by-step
3 # > [reset] Observation: "计算 1+1=?"
4 # > [step 1] Action: "2" → Observation: "" → Reward: 1.0 → Terminated: True
5
6 # 详细日志模式 (RUST_LOG 控制级别)
7 RUST_LOG=uenv=debug uenv debug --env-type math --input debug.json
8 # 输出每步的完整 env.step() 输入输出
9
10 # 重放 EpisodeRequest (从文件)
11 uenv debug --replay saved_request.pb
12 # 使用保存的 EpisodeRequest 重放执行过程
13
14 # 分布式追踪 (OpenTelemetry + tokio-console)
15 RUST_LOG=uenv=trace tokio-console attach http://localhost:6669
16 # 实时查看每个 Episode 的 async span 执行时间

```

**tracing span 示例：**

代码块

```

1 // Worker 中自动生成的 tracing span 树
2 //
3 // dispatch_episode{episode_id="ep-abc123"}
4 // └─ env.reset{seed=42} 耗时: 5ms
5 // └─ step[0] 耗时: 105ms
6 // └─ model.generate 耗时: 100ms
7 // └─ env.step 耗时: 5ms

```

```
8 // |— step[1] 耗时: 110ms
9 // | |— model.generate 耗时: 102ms
10 // | └─ env.step 耗时: 8ms
11 // └─ ...
```

错误诊断:

代码块

```
1 # 查看 Episode 详情
2 uenv status --episode ep-abc123
3 # Episode: ep-abc123
4 # Status: FAILED
5 # Error: ENV_ERROR.TOOL_FAILED
6 # Message: "Code execution timed out after 30s"
7 # Suggestion: "Increase timeout_seconds or simplify the code"
8 # Worker: worker-1
9 # Duration: 30.5s
10 # Trace ID: trace-xyz789
11
12 # 查看 Worker 详情
13 uenv worker status worker-1
14 # Worker: worker-1
15 # Status: ACTIVE
16 # Active Episodes: 3/8
17 # Completed: 150
18 # Avg Duration: 2.3s
19 # Last Heartbeat: 2s ago
20 # Supported: [math, code]
```

16.5 错误码参考

| 错误码                     | 消息      | 可能原因           | 建议修复                   |
|-------------------------|---------|----------------|------------------------|
| ENV_ERROR.INVALID_STATE | 环境状态不正确 | reset 前调用 step | 确保先调用 reset()          |
| ENV_ERROR.TOOL_FAILED   | 工具调用失败  | 工具实现异常         | 检查工具代码，用 uenv debug 调试 |
| ENV_ERROR.PARSE_ERROR   | 输出解析失败  | 环境输出格式不符       | 检查 Observation 格式      |
|                         |         |                |                        |

|                              |              |                     |                        |
|------------------------------|--------------|---------------------|------------------------|
| MODEL_ERROR.UNREACHABLE      | 模型端点不可达      | URL 错误或服务宕机         | 检查 model_endpoint 配置   |
| MODEL_ERROR.OOM              | 模型推理 OOM     | 请求过长                | 减小 max_steps 或分片       |
| MODEL_ERROR.INVALID_RESPONSE | 模型返回格式异常     | API 版本不匹配           | 检查模型 API 版本            |
| TIMEOUT_ERROR.EPISODE        | Episode 整体超时 | 环境执行过慢              | 增大 timeout_seconds     |
| TIMEOUT_ERROR.MODEL_CALL     | 模型调用超时       | 模型推理慢               | 检查模型负载和 GPU 资源         |
| RESOURCE_ERROR.WORKER_FULL   | Worker 已满    | max_concurrent 达上限  | 增加 Worker 数量           |
| RESOURCE_ERROR.NO_WORKER     | 无可用 Worker   | env_type 无注册        | 检查 Worker env_types 配置 |
| CONFIG_ERROR.MISSING_FIELD   | 缺少必需字段       | EpisodeRequest 不完整  | 检查 env_config 结构       |
| PROTOCOL_ERROR.VERSION       | 协议版本不兼容      | Client/Server 版本不匹配 | 升级客户端或服务端              |
| INTERNAL_ERROR.PANIC         | 内部错误         | Worker/Server 崩溃    | 查看日志，提交 issue          |
| INTERNAL_ERROR.WAL           | WAL 写入失败     | 磁盘满或权限不足            | 检查磁盘空间和权限              |

## 十七、部署与配置

本章提供 UEnv 从本地开发到生产部署的完整指南。核心设计思想是"渐进部署"

### 17.1 系统要求

| 场景   | 最低要求                         | 推荐配置           |
|------|------------------------------|----------------|
| 本地开发 | PRust 2024 +, 4 GB RAM, 2 核  | 8 GB RAM, 4 核  |
| 单机容器 | Podman/Podman, 8 GB RAM, 4 核 | 16 GB RAM, 8 核 |

## 17.2 本地开发部署（ProcessBackend）

Podman-compose.yml（开发环境）：

代码块

```
1 version: "3.8"
2 services:
3 uenv-server:
4 image: uenv-server:latest
5 ports:
6 - "50051:50051" # gRPC
7 - "9090:9090" # Metrics
8 environment:
9 - UENV_LOG_LEVEL=INFO
10 - UENV_AUTH_MODE=token
11 - UENV_AUTH_TOKEN=dev-token-change-me
12 volumes:
13 - ./config:/etc/uenv/config
14 healthcheck:
15 test: ["CMD", "curl", "-f", "http://localhost:9090/healthz"]
16 interval: 10s
17 timeout: 5s
18 retries: 3
19
20 uenv-worker:
21 image: uenv-worker:latest
22 depends_on:
23 uenv-server:
24 condition: service_healthy
25 environment:
26 - UENV_SERVER_ENDPOINT=uenv-server:50051
27 - UENV_ENV_TYPES=math,code
28 - UENV_MAX_CONCURRENT=4
29 - UENV_AUTH_TOKEN=dev-token-change-me
30 deploy:
31 replicas: 2
32
33 prometheus:
34 image: prom/prometheus:latest
35 ports:
36 - "9091:9090"
37 volumes:
38 - ./prometheus.yml:/etc/prometheus/prometheus.yml
39
40 grafana:
```

```
41 image: grafana/grafana:latest
42 ports:
43 - "3000:3000"
44 environment:
45 - GF_SECURITY_ADMIN_PASSWORD=admin
```

### 一键启动：

Code block

```
1 Podman-compose up -d
2 uenv status # 验证系统就绪
```

## 17.3 Podman 单机部署

### systemd unit 文件：

代码块

```
1 # /etc/systemd/system/uenv-server.service
2 [Unit]
3 Description=UEnv Server
4 After=network.target
5
6 [Service]
7 Type=simple
8 ExecStart=/usr/bin/podman run --rm \
9 --name uenv-server \
10 -p 50051:50051 -p 9090:9090 \
11 -v /etc/uenv/config:/etc/uenv/config:ro \
12 -v /var/lib/uenv/wal:/var/lib/uenv/wal \
13 -e UENV_LOG_LEVEL=INFO \
14 uenv-server:latest
15 Restart=on-failure
16 RestartSec=5
17
18 [Install]
19 WantedBy=multi-user.target
```

## 17.4 配置参考

### UEnv Server 配置项：

| 配置项                           | 类型     | 默认值               | 说明                            |
|-------------------------------|--------|-------------------|-------------------------------|
| UENV_GRPC_PORT                | int    | 50051             | gRPC 监听端口                     |
| UENV_METRICS_PORT             | int    | 9090              | Prometheus metrics 端口         |
| UENV_LOG_LEVEL                | string | INFO              | 日志级别                          |
| UENV_LOG_FORMAT               | string | json              | 日志格式 (json/text)              |
| UENV_AUTH_MODE                | string | token             | 认证模式<br>(token/mtls/disabled) |
| UENV_AUTH_TOKEN               | string | —                 | Token 认证的共享密钥                 |
| UENV_TLS_CERT_PATH            | string | —                 | TLS 证书路径                      |
| UENV_TLS_KEY_PATH             | string | —                 | TLS 私钥路径                      |
| UENV_MAX_MESSAGE_SIZE         | string | 64Mi              | gRPC 最大消息大小                   |
| UENV_SCHEDULER_QUEUE_SIZE     | int    | 10000             | 调度队列最大深度                      |
| UENV_HEARTBEAT_TIMEOUT_FACTOR | int    | 3                 | 心跳超时倍数                        |
| UENV_WAL_DIR                  | string | /var/lib/uenv/wal | WAL 目录                        |
| UENV_WAL_ENABLED              | bool   | true              | 是否启用 WAL 持久化                  |

### Worker 配置项：

| 配置项                   | 类型     | 默认值             | 说明             |
|-----------------------|--------|-----------------|----------------|
| UENV_SERVER_ENDPOINT  | string | localhost:50051 | Server gRPC 地址 |
| UENV_ENV_TYPES        | string | —               | 支持的环境类型 (逗号分隔) |
| UENV_MAX_CONCURRENT   | int    | 4               | 最大并发 Episode 数 |
| UENV_WARMUP_POOL_SIZE | int    | 2               | 每环境类型预热实例数     |
|                       | int    | 1000            | 单实例最大复用次数      |

|                            |        |               |                      |
|----------------------------|--------|---------------|----------------------|
| UENV_MAX_EPISODE_COUNT     |        |               |                      |
| UENV_HEARTBEAT_INTERVAL    | int    | 5             | 心跳间隔（秒）              |
| UENV_WORKER_ID             | string | auto          | Worker 标识（auto=自动生成） |
| UENV_BACKEND               | string | process       | 后端类型                 |
| UENV_AUTH_TOKEN            | string | —             | 认证 Token             |
| UENV_TLS_CA_PATH           | string | —             | CA 证书路径              |
| UENV_WAL_DIR               | string | /tmp/uenv/wal | 本地 WAL 目录            |
| UENV_RECONNECT_MAX_BACKOFF | int    | 30            | 重连最大退避（秒）            |

配置优先级：

代码块

1 CLI 参数 > 环境变量 > ConfigMap > 配置文件 > 默认值

17.5 网络规划

| 端口    | 协议   | 用途               | 来源               |
|-------|------|------------------|------------------|
| 50051 | gRPC | Server 主服务       | Adapter, Worker  |
| 9090  | HTTP | Metrics + Health | Prometheus, 健康检查 |
| 3000  | HTTP | Grafana（可选）      | 运维浏览器            |

防火墙规则：

代码块

1 ALLOW Adapter → Server:50051 (gRPC)  
2 ALLOW Worker → Server:50051 (gRPC)  
3 ALLOW Prometheus → Server:9090 (HTTP)  
4 ALLOW Worker → Model Endpoint:443 (HTTPS)



## 十八、迁移指南

### 18.1 版本变更摘要

### 18.2 不兼容变更清单

### 18.3 渐进迁移路径

### 18.4 环境迁移指南

### 18.5 训练框架迁移指南

### 18.6 回滚策略

---